OpenSSL API Compatibility Wrapper - QSCOSSL

Revision 1.0.0, November 8, 2026

John G. Underhill – Quantum Resistant Cryptographic Solutions Corporation

This document is an engineering-level description of the QSCOSSL OpenSSL source-compatibility wrapper. It explains the integration model, exported interfaces, object ownership, cryptographic and TLS usage, build configurations, security limitations, test evidence, and implementation references.

The QSCOSSL software and its documentation are proprietary QRCS materials, licensed under the QRCS Commercial Application Software License (QRCS-CASL).

Contents **Page**

1 Foreword 3
2 Introduction 4
3 References 5
4 Tables and Terminology 6
5 Compatibility Architecture 7
6 Build, Configuration and Linking 9
7 Core Runtime, Memory and Errors 11
8 Entropy and Random Providers 12
9 Digests, HMAC and MAC 13
10 EVP Symmetric Ciphers and AEAD 15
11 Key Derivation and XOF 17
12 Classical Public-Key Operations 18
13 Post-Quantum Cryptography 19
14 Certificates, X.509 and Revocation 21
15 TLS 1.3 Client and Server 23
16 OpenSSL 3 Provider and Hardware Adapters 25
17 Integration Examples and Migration Patterns 27
18 Testing, Security Review and Limitations 28

19 Public Header API Reference 31
20 Implementation and Release Checklist 73
Annex A Data Types and Constants 74
Annex B Repository and Test Index 75

1: Foreword

This specification is intended for C and C++ implementors migrating selected OpenSSL-source applications to QSC. It describes the QSCOSSL compatibility library exactly as exposed in the supplied QSCOSSL(2).zip project snapshot, not as a guarantee of complete OpenSSL parity.

The QSCOSSL headers contain deliberately familiar names from OpenSSL, but the objects and implementations are QSC-backed. QSCOSSL adds separately named QSCOSSL_* routines where a stock OpenSSL signature cannot express an essential safety contract, such as authenticated plaintext output capacity, independent ML-KEM parameter selection or authenticated revocation status.

This paper follows the structure of the QRCS QSC library specification: sections, descriptions, tables, call hierarchies, explicit API declaration references, notes and normative references. Original function names are preserved. Details that depend on an optional build feature, separate provider or external QSC dependency are clearly identified.

2: Introduction

QSCOSSL is a C-language translation and migration layer for an intentional subset of the OpenSSL EVP, HMAC, RAND, BIO, PEM, X.509 and SSL/TLS APIs. Its purpose is to allow selected OpenSSL-oriented source code to be recompiled using QSC cryptography, certificate infrastructure and a QSC TLS 1.3 stack, while retaining recognizable initialization, update and finalization patterns.

The wrapper is not libcrypto/libssl and does not reproduce the complete OpenSSL object model, dispatch system, supported curves, algorithms, protocol versions or binary interface. The code is not a drop-in replacement for precompiled OpenSSL applications. Porting requires an API inventory, source changes for QSCOSSL-specific functions, coherent builds, and application-specific qualification.

The separately built OpenSSL 3 provider is an independent integration route: real OpenSSL applications fetch selected QSC-backed algorithm implementations through OpenSSL 3 provider facilities while continuing to use the genuine OpenSSL libraries and headers. A program should not conflate this route with the QSCOSSL replacement-header compatibility library.

Document baseline:

Property	Specification
Wrapper source	QSCOSSL(2).zip; README development integration dated November 8, 2026
Underlying QSC	External QSC library snapshot referenced as QSC 2026-11-06-B, including required TLS/OCSP fixes
Wrapper compatibility version	QSCOSSL_VERSION_TEXT remains "0.1.0-stage10" in qscossl_config.h; development metadata is newer
License	QRCS Commercial Application Software License (QRCS-CASL); controlling terms supplied separately
Integration modes	Replacement OpenSSL-style headers with QSCOSSL static library; optional genuine OpenSSL 3 provider
Release posture	Development / targeted source migration; no FIPS/CMVP validation claimed

Table 2.1 QSCOSSL source baseline

3: References

3.1 Project and architecture references

The following sources were examined within the supplied QSCOSSL project. The exact headers supersede older narrative notes when the two differ.

- README.md — Development status, build options, compatibility boundaries and current qualification summary.
- QSCOSSL/openssl/*.h — OpenSSL-style public API declarations and Doxygen contract notes.
- QSCOSSL/qscossl*.h — QSCOSSL extensions, sizes, error enums, hardware and revocation interfaces.
- examples/*.c — Nine independent implementor examples.
- QSCOSSLTest/*.c — Stage-oriented test source and direct named test functions.
- docs/QSCOSSL Implementor Integration and API Guide.md — Integration, ownership and security guidance.
- docs/QSCOSSL API Compatibility Matrix.md — Compatibility/unsupported-call classification.
- docs/QSCOSSL Stages 30-32 Qualification 2026-11-07.md — Latest dated certificate/TLS/qualification discussion; historical results distinguished.
- CMakeLists.txt / QSCOSSL.sln / QSCOSSL.vcxproj — Actual build configuration and Windows project definitions.

3.2 Normative and external references

QSC library primary repository https://github.com/QRCS-CORP/QSC

OpenSSL project reference documentation https://docs.openssl.org/3.5/

OpenSSL 3 provider architecture https://docs.openssl.org/3.0/man7/provider/

OpenSSL EVP cipher APIs https://docs.openssl.org/3.5/man3/EVP_EncryptInit/

RFC 8446: TLS 1.3 https://www.rfc-editor.org/rfc/rfc8446

RFC 5869: HKDF https://www.rfc-editor.org/rfc/rfc5869

NIST SP 800-185: cSHAKE and KMAC https://doi.org/10.6028/NIST.SP.800-185

NIST FIPS 203: ML-KEM https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204: ML-DSA https://doi.org/10.6028/NIST.FIPS.204

RFC 5280: X.509 PKI https://www.rfc-editor.org/rfc/rfc5280

RFC 6960: OCSP https://www.rfc-editor.org/rfc/rfc6960

4: Tables and Terminology

4.1 Terms and units

Term	Definition
ABI	Binary interface: layout and calling conventions of compiled libraries. QSCOSSL does not replace the OpenSSL ABI.
API	Callable source interface through the installed QSCOSSL headers.
AEAD	Authenticated encryption with associated data; releases plaintext only after tag verification.
AAD	Additional authenticated data, authenticated but not encrypted.
CSP / ACP	QSC system cryptographic service provider and automated entropy collection provider.
KEM	Key encapsulation mechanism; public encapsulation and private decapsulation.
Opaque object	A pointer-only handle allocated by a wrapper constructor; do not allocate by sizeof or copy.
Borrowed pointer	Object/data owned by another interface or caller; do not free unless an API explicitly transfers ownership.
Capacity	Writable number of bytes allocated; separate from current output length.
Derivation context	Salt, info and personalization values used to separate derived cryptographic keys.

Table 4.1 Terminology

Unless otherwise identified, size_t and integer length parameters in this paper are byte counts, not bit counts. Output lengths set by wrappers must be checked before the output is consumed. Error enums and protocol identifiers are distinct from success/failure status codes.

4.2 Function contract notation

The API pages use the original QSC specification format: Header, Description, Structures, Constants, Call Hierarchy, API and References. The appendix provides the signatures detected in the supplied public headers. The signature is the source of truth for parameter order and types; any generic Doxygen summary should be interpreted together with implementation-specific notes.

5: Compatibility Architecture

5.1 Two distinct integration paths

Compatibility library	OpenSSL 3 provider module
Application includes QSCOSSL-provided openssl/*.h.	Application includes genuine OpenSSL 3 headers.
Links qscossl and the matching QSC library.	Loads qscossl_provider module through real OpenSSL 3.
Own opaque QSCOSSL compatibility contexts.	Use real EVP/provider objects and OSSL_DISPATCH.
Selected OpenSSL-like EVP, SSL, X.509, RAND interfaces.	Selected SHA-2/3, SHAKE, MAC, HKDF, PBKDF2 and cipher implementations.
No binary compatibility with OpenSSL.	Real OpenSSL provider dispatch; not a full QSCOSSL compatibility API.

Table 5.1 Integration route comparison

Do not build one application translation unit against both incompatible definitions of the same OpenSSL public structures. If a system must use both integrations, isolate the libraries and namespaces by audited architecture, build steps and linkage, rather than mixing headers.

5.2 Object lifetime and ownership

Object family	Creation	Release / policy
EVP_CIPHER_CTX	EVP_CIPHER_CTX_new	EVP_CIPHER_CTX_free; reset permits reuse; do not share mutation.
EVP_MD_CTX	EVP_MD_CTX_new	EVP_MD_CTX_free; reset clears digest state.
HMAC_CTX	HMAC_CTX_new	HMAC_CTX_free; key data is borrowed when passed for init.
EVP_PKEY, EVP_PKEY_CTX	new/raw constructors, keygen, context constructors	Matching *_free; exported raw bytes remain caller-owned.
BIO	BIO_new_file / BIO_new_mem_buf	BIO_free_all; monitor file/memory ownership semantics.
X509, X509_STORE(_CTX)	X509_new/d2i/PEM; STORE constructors	Independent frees; store copies anchors and verification context copies intermediates.
SSL_CTX and SSL	SSL_CTX_new / SSL_new	SSL_free / SSL_CTX_free; borrowed TCP sockets remain caller-owned.
QSCOSSL_MAC_CTX	QSCOSSL_MAC_CTX_new	QSCOSSL_MAC_CTX_free; do not share mutable contexts.

Table 5.2 Owner objects

5.3 Key separation and policy

A source-level similarity is not a security-policy equivalence. Some EVP operations return DER-encoded signatures while QSC-native signatures use raw representations; a cipher descriptor is not proof of AEAD authentication; parsing X.509 is not trust validation; and signed revocation evidence is meaningful only after issuer trust is independently established.

Opaque key objects encapsulate QSC state, but exported key bytes, PKCS#8 material and encapsulated shared secrets remain subject to the caller's own storage, protection, erasure and access-control requirements.

5.4 Unsupported and fail-closed operation categories

Legacy RSA/DSA/finite-field DH, BIGNUM, ENGINE and low-level EC_KEY compatibility are outside the selected source wrapper. BIO-driven/nonblocking TLS and full OpenSSL API surface are not implemented. PKCS#12 export, full online OCSP fetching/stapling, full CA issuance and provider-side AEAD/asymmetric dispatch are not implemented. Where a selector is defined only for compatibility, successful compilation is not proof that its operation is supported.

5.5 Interface audit and compatibility decisions

Before editing source, inventory every OpenSSL include directive and each externally linked OpenSSL symbol in the application. Categorize calls as directly supported, requiring a QSCOSSL extension, provided only through the separate OpenSSL 3 provider, or unsupported. This prevents a successful compile from being mistaken for complete semantic compatibility.

Give special attention to functions that carry hidden capacity or object-lifetime assumptions. In QSCOSSL, an EVP cipher descriptor is borrowed and remains valid without release; the context created for a cipher operation is owned and must be freed. The analogous distinctions apply to EVP message-digest handles, X.509 certificates, stores, and SSL connection objects.

Do not map an unsupported OpenSSL operation to a similarly named alternative merely to satisfy the linker. The engineer must document any required application redesign,

including authentication policy, error treatment, concurrency constraints, and the format of network or stored cryptographic objects.

6: Build, Configuration and Linking

6.1 Repository structure

```
QSC/                    [external dependency; independently built]
QSCOSSL/
  QSCOSSL.sln           [Visual Studio solution]
  QSCOSSL.vcxproj       [static wrapper]
  QSCOSSL/openssl/      [compatibility public headers]
  QSCOSSL/qscossl*.h    [extension public headers]
  QSCOSSL/qscossl*.c    [implementation and optional provider]
  QSCOSSLTest/          [named stage tests]
  examples/             [standalone examples]
  docs/                 [qualification and migration notes]
```

6.2 Windows / Visual Studio

Use the supplied Visual Studio 2022 solution and ensure QSC is built first. Compile the wrapper with the same architecture, CRT setting, debug/release mode and chosen AES/SIMD profile as its external QSC dependency. The project contains a static-library target and a test executable; its presence in the ZIP is not a claim that this specification-generation environment rebuilt it.

The current wrapper can be built on a machine without AVX-512 execution capability when targeting such instructions, but an AVX-512 binary must not be executed on an unsupported CPU. Profile compatibility between caller, wrapper and QSC remains essential because internal structures and implementations can be build-sensitive.

6.3 Portable CMake configuration

```
cmake -S . -B build -G Ninja \
  -DQSCOSSL_QSC_INCLUDE_DIR=/path/to/QSC \
  -DQSCOSSL_QSC_LIBRARY=/path/to/libqsc.a \
  -DQSCOSSL_BUILD_TESTS=ON \
  -DQSCOSSL_BUILD_EXAMPLES=ON
cmake --build build
```

Set QSCOSSL_QSC_LIBRARY to a compatible .lib when using MSVC. The include path must point to the public headers of the matching external QSC revision. The build fails configuration when required dependency locations are not supplied for linked test/example targets.

6.4 Configuration switches

CMake setting	Default	Implementor implication
QSCOSSL_BUILD_TESTS	ON	Build the test executables.
QSCOSSL_BUILD_EXAMPLES	OFF	Build nine standalone examples.
QSCOSSL_BUILD_NETWORK_TESTS	OFF	Enable live loopback TLS suites with system network prerequisites.
QSCOSSL_ENABLE_EXTENDED_CIPHERS	ON	Requires RCS/CSX implementation objects in QSC.
QSCOSSL_ENABLE_INDEPENDENT_MLKEM	ON	Requires all independent QSC Kyber/ML-KEM implementations.
QSCOSSL_BUILD_OPENSSL3_PROVIDER	OFF	Build standalone real OpenSSL 3 provider; QSC archive must be PIC for ELF module.

Table 6.1 Build switches

6.5 Symbol mismatch and packaging diagnostics

Link failures for QSC TLS socket-attachment or revocation helpers often indicate that the external QSC revision predates the required dependency repairs. Do not patch QSC or disable certificate/TLS verification simply to make the wrapper link. Resolve dependency versioning and rerun qualification. The deployment package must contain the correct QSC runtime/static archive and provider module only where those modes were intentionally selected.

6.6 Build artifact and CPU-profile acceptance

Use exactly one selected QSC compilation profile per linked wrapper binary. Confirm that caller and library configurations agree on architecture, AES instruction selection, and runtime ABI. The installed profile must be legal for the target CPU; compiling an AVX-512 translation unit does not imply that every deployment processor can execute it.

For each packaged platform, record the actual QSC and QSCOSSL revisions, compiler and linker versions, warning level, build definitions, dependency paths, and binary hashes. Qualify the static wrapper and any dynamically loaded OpenSSL provider separately; the latter additionally depends on the genuine OpenSSL 3 development ABI.

A release package should include its permitted redistributables, matching public headers, target-specific application examples, relevant QRCS commercial license material, and a reproducible native test record. External QSC sources remain an

independently managed dependency and should not be modified as part of wrapper packaging.

7: Core Runtime, Memory and Errors

7.1 Initialization and version

Call qscossl_initialize or the appropriate OpenSSL-style OPENSSL_init_crypto/OPENSSL_init_ssl entry point before subsystem use. Inspect status. qscossl_version_number and qscossl_version_text identify the selected compatibility layer; their values are not the version number of real OpenSSL. The source header currently retains a stage10 version string alongside newer development integration notes.

7.2 Memory operations

OPENSSL_malloc/zalloc/realloc/strdup/free are compatibility allocation functions. OPENSSL_clear_free and OPENSSL_secure_clear_free erase owned buffers where documented before release. OPENSSL_cleanse is provided for secret wiping. Do not apply an OpenSSL wrapper allocator to QSC-owned opaque objects, or free borrowed descriptors.

CRYPTO_memcmp is available for timing-resistant byte comparison. Equal-length checks and application error paths remain the caller's responsibility; a constant-time comparison does not turn a digest into a MAC.

7.3 Error reporting

QSCOSSL exposes OpenSSL-style ERR_clear_error, ERR_get_error, ERR_peek_error, ERR_reason_error_string, ERR_error_string_n and ERR_put_error, together with qscossl_error_* functions. Capture errors promptly following a failed call; processing a later operation may replace or clear the current diagnostic.

```
if (EVP_DigestInit_ex(context, EVP_sha256(), NULL) != 1)
{
    const unsigned long error = ERR_get_error();
    printf("QSCOSSL digest initialization failed: %s\n",
        ERR_reason_error_string(error));
}
```

The above pattern is illustrative; refer to examples/evp_digest_example.c for the complete allocation, cleanup and output logic. Do not use diagnostic strings as stable machine-readable error categories.

7.4 State cleanup and error containment

An application should establish a single cleanup path for each operation. On an allocation, initialization, update, finalization, or verification failure, capture the reported error promptly, avoid using output that may be partial, and release every successfully constructed owned object in reverse order.

OPENSSL_cleanse is intended to reduce the lifetime of secrets in caller-owned memory. Clear buffers containing long-term secret keys, password-derived keys, intermediate shared secrets, and serialized private keys when no longer required. Wiping an application buffer does not release an opaque wrapper object, nor does freeing a wrapper object necessarily sanitize unrelated caller memory.

Error strings explain what happened but must not be used as stable machine-readable policy codes. Build higher-level application status from documented numeric results and state transitions. Never downgrade a failed authenticated operation, certificate validation, or hardware-key requirement to an unprotected fallback.

8: Entropy and Random Providers

QSCOSSL delegates random generation to the QSC CSP or ACP provider models. The compatibility RAND layer does not expose QSC SecRand. The recommended baseline is CSP unless project-specific policy requires the alternate ACP provider.

8.1 Selection and call sequence

```
QSCOSSL_RAND_set_provider(QSCOSSL_RAND_PROVIDER_CSP);
RAND_status();
RAND_bytes(buffer, bufferlength);
RAND_priv_bytes(secretbuffer, secretlength);
```

Check status before consuming random output. Recheck after failures, and do not treat a deterministic test vector or zero-filled buffer as production entropy. The RAND wrappers accept integer byte counts; ensure requested lengths fit their public parameter types.

8.2 Availability and failure policy

RAND_poll and RAND_status expose the current provider readiness model; a successful status reflects this source wrapper's provider interface, not a guarantee of independent entropy certification. Key generation depends on external QSC provider availability. See openssl/rand.h and examples/rand_example.c.

9: Digests, HMAC and MAC

9.1 EVP digest model

The EVP digest API provides SHA-256, SHA-384, SHA-512, SHA3-256 and SHA3-512 descriptors. A descriptor returned by EVP_sha*() is borrowed. The EVP_MD_CTX_new allocator returns an owned context. Perform initialization, arbitrary-length incremental updates and finalization, then free the context.

```
EVP_MD_CTX* context = EVP_MD_CTX_new();
EVP_DigestInit_ex(context, EVP_sha3_512(), NULL);
EVP_DigestUpdate(context, message, messagelen);
EVP_DigestFinal_ex(context, digest, &digestlength);
EVP_MD_CTX_free(context);
```

Algorithm	Output size	Descriptor
SHA-256	32 bytes	EVP_sha256()
SHA-384	48 bytes	EVP_sha384()
SHA-512	64 bytes	EVP_sha512()
SHA3-256	32 bytes	EVP_sha3_256()
SHA3-512	64 bytes	EVP_sha3_512()

Table 9.1 Supported digest selectors

9.2 HMAC and QSC MAC extensions

OpenSSL HMAC compatibility targets HMAC-SHA2-256/384/512. HMAC over SHA-3 is not mapped to the HMAC compatibility path; use QSCOSSL MAC extensions for KMAC-256, KMAC-512, QMAC or Poly1305 where policy permits. These are distinct constructions and must not be substituted without a protocol change.

For HMAC, construct HMAC_CTX_new, select a supported digest in HMAC_Init_ex, update all bytes and finalize into an output buffer sized by the selected digest. For QSCOSSL_MAC_CTX, select a qscossl_algorithm_* identifier and call QSCOSSL_MAC_init,

update, final and free. Never reuse a Poly1305 one-time key across messages. Verify MACs using a constant-time comparator.

9.3 Call hierarchy

```
HMAC_CTX_new -> HMAC_Init_ex -> HMAC_Update -> HMAC_Final -> HMAC_CTX_free
QSCOSSL_MAC_CTX_new -> QSCOSSL_MAC_init -> QSCOSSL_MAC_update
  -> QSCOSSL_MAC_final -> QSCOSSL_MAC_CTX_free
```

9.4 Hash and MAC selection guidance

The message digest APIs provide deterministic message fingerprints; they do not authenticate sender identity. For a keyed integrity check, use a suitable HMAC, KMAC or other explicitly authenticated construction, and compare verification results without leaking secret-dependent comparison information.

An incremental digest or HMAC computation must be initialized for the selected algorithm and key, then supplied with message fragments in the same order used by the protocol. The final result must be produced into an appropriately sized caller buffer and the output length must be checked before transmitting or storing the value.

For MACs with customization parameters, encode and document the agreed domain-separation string. A KMAC customization value is not a textual label that can be changed freely after deployment: peers verifying the same message must use the same key, customization, output length and byte-level message representation.

10: EVP Symmetric Ciphers and AEAD

10.1 Supported selector families

Cipher mode	Public selector family	Implementation note
AES ECB	EVP_aes_128_ecb, EVP_aes_256_ecb	Testing/internal block utility only; not suitable for general data protection.
AES CBC	EVP_aes_128_cbc, EVP_aes_256_cbc	PKCS#7 padding defaults on; does not authenticate ciphertext.
AES CTR	EVP_aes_128_ctr, EVP_aes_256_ctr	Byte-granular stream operation; key+counter IV reuse forbidden.
AES GCM	EVP_aes_128_gcm, EVP_aes_256_gcm	AEAD; application supplies/gets authentication tag.
ChaCha20	EVP_chacha20	Unauthenticated stream cipher; must protect integrity separately.

Cipher mode	Public selector family	Implementation note
ChaCha20-Poly1305	EVP_chacha20_poly1305	AEAD; plaintext withheld until verified finalizer succeeds.

Table 10.1 Supported symmetric descriptors

10.2 Traditional EVP transform

For CBC, CTR and ECB, the standard sequence is context allocation, EVP_EncryptInit_ex/EVP_DecryptInit_ex, one or more EVP_*Update calls, corresponding EVP_*Final_ex, and release. The actual byte count for every Update and Final must be accumulated; output capacity must account for padding or the mode-specific block size.

```
EVP_CIPHER_CTX_new
  -> EVP_EncryptInit_ex / EVP_DecryptInit_ex
  -> EVP_EncryptUpdate / EVP_DecryptUpdate [repeat as needed]
  -> EVP_EncryptFinal_ex / EVP_DecryptFinal_ex
  -> EVP_CIPHER_CTX_free
```

AES-CTR encryption and decryption both invoke forward AES block encryption internally to generate counter keystream, even when the caller selects decryption. QSCOSSL corrects this direction for the QSC AES-NI backend. Ensure the linked dependency includes the corresponding fix and test both directions.

10.3 Authenticated encryption contract

An AEAD encryption operation provides key, unique nonce, optionally associated data, payload and output tag. AAD is passed through EVP_EncryptUpdate with NULL output before payload. After finalization, use EVP_CIPHER_CTX_ctrl GET_TAG to obtain the authenticator. Decryption supplies AAD and ciphertext, sets the expected tag, and finalizes only through the capacity-aware QSCOSSL_EVP_DecryptFinal_ex2 function.

The wrapper buffers authenticated decryption output, and intentionally refuses to release that buffered plaintext through legacy EVP_DecryptFinal_ex because the latter does not accept an output capacity. This is a deliberate compatibility change: a successful migration must call the QSCOSSL-specific finalizer with the actual available writable bytes. On tag failure, do not use any provisional result.

```
EVP_DecryptInit_ex(dec, EVP_aes_256_gcm(), NULL, key, nonce);
EVP_DecryptUpdate(dec, NULL, &aadlen, aad, aadlength);
EVP_DecryptUpdate(dec, output, &cipherbytes, ciphertext, ciphertextlength);
EVP_CIPHER_CTX_ctrl(dec, EVP_CTRL_GCM_SET_TAG, tagsize, tag);
QSCOSSL_EVP_DecryptFinal_ex2(dec, output, outputcapacity, &plaintextlength);
```

Note: This is a call-sequence extract illustrating the wrapper-specific finalization rule. Check every return value and allocate all objects and buffers as demonstrated in examples/evp_aead_example.c. The correct output and update mode should be taken from that complete example and the current source header contract.

10.4 RCS and CSX extensions

The qscossl_extcipher.h adapters expose one-shot authenticated QSC RCS-256/RCS-512 and CSX-512 operations with explicit output capacity, output length and optional associated/customization strings. They are conditional on QSCOSSL_EXTENDED_CIPHERS_ENABLED and matching QSC implementation objects. Use QSCOSSL_EXTCIPHER_is_available before selecting them. These are not OpenSSL EVP cipher identifiers.

QSCOSSL_RCS_seal and QSCOSSL_CSX_seal produce ciphertext plus a tag; corresponding open operations authenticate before exposing plaintext. Use the key/nonces and exact tag sizes selected by the QSC profile. Reusing a nonce under the same key is a security failure even when a one-shot wrapper makes state handling convenient.

10.5 AEAD verification and destination-capacity policy

Authenticated decryption is intentionally stricter than classic EVP decryption. The wrapper buffers protected ciphertext during update and requires QSCOSSL_EVP_DecryptFinal_ex2 to verify the configured tag and to receive the actual writable plaintext capacity. Its return status and authenticated byte count are the only authorization to release plaintext to another component.

On a failed tag check, insufficient destination capacity or invalid context, the caller must treat the operation as failed and discard that record. A decryption buffer may contain stale or unrelated data; the caller must never infer validity from an unchanged buffer. The capacity supplied to the finalizer is a byte count, not an assumption derived from ciphertext length.

CTR and CBC compatibility do not automatically provide a cryptographic integrity guarantee. The calling protocol must specify an independent authentication construction, nonce/IV management, ordering, and replay policy. For GCM or

ChaCha20-Poly1305, key/nonce reuse is prohibited by the security contract; review every error and retry path for accidental reuse.

11: Key Derivation and XOF

11.1 HKDF

QSCOSSL_KDF_hkdf implements RFC 5869 key derivation for SHA-256, SHA-384 or SHA-512. Its explicit parameters are digest, salt and salt length, input key material and length, info and length, output and requested length. It returns a checked status, clears output on failure and enforces RFC 5869 maximum output of 255 digest blocks.

Use distinct application-context info values for unrelated purposes. Salt and info are not interchangeable; neither turns a low-entropy password into an appropriate HKDF input secret.

11.2 PBKDF2 and password handling

The QSCOSSL_KDF_pbkdf2 direct helper uses QSC PBMAC1 and supports derived lengths up to the documented 64-byte maximum. The OpenSSL-style PKCS5_PBKDF2_HMAC entry point handles longer derivations. Choose high-entropy salt, a suitable work factor and a defensible password policy. Check success and erase derived secret material after use.

11.3 SHAKE and cSHAKE

QSCOSSL_SHAKE_compute and QSCOSSL_cSHAKE_compute expose SHAKE128/256 and cSHAKE128/256 with explicit output byte length. cSHAKE receives separate function-name and customization byte strings for domain separation. These operations may be suitable as subcomponents of standardized constructions, but do not substitute a custom ad hoc XOF-based protocol for a specified KDF.

11.4 KDF interoperability and secret lifetime

HKDF salt, input key material, and application-specific info have separate roles. The same input secret can safely derive distinct purpose-labeled outputs only when each protocol independently defines the output lengths and domain-separation encoding. Keys generated for one algorithm or role should not be silently reused for a different one.

PBKDF2 is intended for password-derived material and must use a deployment-appropriate work factor and unpredictable salt; it is not a substitute for a high-entropy KEM or ECDH shared secret. The correct work factor is application and threat-model dependent, so the paper does not prescribe a fabricated universal iteration count.

After deriving session keys, erase temporary shared secrets and derivation buffers according to the caller's ownership model. Persisted private keys, exported raw key objects and key-wrap material require a distinct storage and access-control policy.

12: Classical Public-Key Operations

12.1 EVP_PKEY identities and lifecycle

The EVP_PKEY compatibility layer supports selected ECDH, ECDSA, EdDH/X25519/X448 and EdDSA/Ed25519/Ed448 operations. Use EVP_PKEY_CTX_new_id and EVP_PKEY_keygen_init/keygen for supported key types; EVP_PKEY_new_raw_private_key/public_key and the matching getters implement selected serialization forms. Distinguish raw curve key formats from traditional SEC1/PKCS#8 file formats.

12.2 Key agreement

For public-key agreement, initialize a derive context, bind the validated peer key, query the expected output size as supported, allocate the output, and invoke EVP_PKEY_derive. The resulting shared secret must be processed by a protocol-defined KDF before use as an application key; never interpret successful key agreement as identity authentication.

12.3 Signature representation

A selected EVP_PKEY_EC signing path uses conventional DER-encoded ECDSA signature representation, whereas the QSC-specific EVP_PKEY_QSC_ECDSA family retains QSC-native raw r || s. Cross-library interoperability requires using the agreed wire encoding. EdDSA signs messages according to its own scheme, not by substituting an unrelated digest policy.

```
EVP_DigestSignInit -> EVP_DigestSignUpdate -> EVP_DigestSignFinal
EVP_DigestVerifyInit -> EVP_DigestVerifyUpdate -> EVP_DigestVerifyFinal
```

The published examples demonstrate ML-DSA detached signing and verification. For any signing flow, verify that the certificate or public key is bound to the intended identity; a mathematically correct signature alone does not establish authorization.

13: Post-Quantum Cryptography

13.1 KEM families

The EVP_PKEY_QSC_MLKEM, EVP_PKEY_QSC_HQC and EVP_PKEY_QSC_MCELIECE identifiers expose selected QSC post-quantum KEM operations. QSCOSSL_KEM_keygen/encapsulate/decapsulate are wrapper extensions, not stock OpenSSL KEM APIs. Query exact buffer sizes from the implemented key type or the QSCOSSL_MLKEM_get_sizes profile helper before allocating memory.

13.2 Independent ML-KEM profiles

Profile	Selection	Policy
ML-KEM-512	qscossl_mlkem_512	NIST standardized parameter set.
ML-KEM-768	qscossl_mlkem_768	NIST standardized parameter set.
ML-KEM-1024	qscossl_mlkem_1024	NIST standardized parameter set.
Kyber-1280 experimental	qscossl_mlkem_1280_experimental	QSC nonstandard experimental profile; not ML-KEM standard.

Table 13.1 ML-KEM runtime profile selection

The independent profile adapter requires explicit QSC compilation support. With compact QSC profiles excluding one or more independent Kyber objects, disable QSCOSSL_ENABLE_INDEPENDENT_MLKEM and do not advertise this adapter. KEM decapsulation return status does not necessarily signal authenticated ciphertext: implicit rejection may still return a derived secret for malformed ciphertext. Authenticate protocol transcripts and peers separately.

13.3 PQC signatures

The wrapper has identifiers for ML-DSA, SPHINCS+ and Falcon; their actual availability depends on matching QSC build content and selected parameter policy. Detached signing is through EVP_DigestSign* and verification through EVP_DigestVerify* compatibility functions. Use exact signature length queries, algorithm-specific key sizes and validated identities. The provided example examples/pqc_kem_sign_example.c demonstrates ML-KEM and ML-DSA flows.

13.4 Parameter selection and wire-format boundaries

Runtime KEM profile selection must be bound into protocol negotiation and acceptance policy; do not permit an attacker to choose a weaker or experimental profile by silently changing an algorithm identifier. The special Kyber-1280 profile mentioned by this project is experimental and must not be labeled as standardized ML-KEM.

KEM shared-secret derivation alone does not authenticate which peer supplied the encapsulation key. Application protocols must authenticate identities and protect transcript integrity, especially where decapsulation uses implicit rejection rather than an explicit ciphertext-validity flag.

Deterministic or random signature encodings must be negotiated and validated at the protocol boundary. The OpenSSL-style ECDSA signature representation is DER in the documented path; raw fixed-width QSC signing outputs are not interchangeable without an explicit conversion contract.

14: Certificates, X.509 and Revocation

14.1 BIO and PEM entry points

BIO_new_file opens a file-backed BIO; BIO_new_mem_buf supplies a memory-backed read source. The caller releases BIO instances using BIO_free_all. PEM_read_bio_X509 and PEM_read_X509 create owned X509 certificate objects. Private-key PEM import accepts supported SEC1/PKCS#8 EC and Ed key representations, including selected PBES2-encrypted PKCS#8 containers. Generic OpenSSL password callbacks and legacy PEM encryption formats remain outside this wrapper.

14.2 Certificate verification

Create a trust store containing independently trusted anchors, then a verification context for the presented leaf. Load needed intermediates through the QSCOSSL_X509_STORE_CTX_add_intermediate deep-copy extension; it is not equivalent to trusting them. Call X509_verify_cert, then X509_check_host for the expected DNS name. Neither parsing a certificate nor merely checking its signature establishes an approved path to an explicitly trusted anchor.

```
BIO_new_file -> PEM_read_bio_X509 -> X509_STORE_new
        -> X509_STORE_load_locations
        -> X509_STORE_CTX_new -> X509_STORE_CTX_init
```

-> [add intermediates] -> X509_verify_cert
-> X509_check_host -> matching free operations

14.3 PKCS#12

QSCOSSL_PKCS12_import_der authenticates a PFX bundle before key and certificate extraction. QSCOSSL_PKCS12_free releases the imported container; QSCOSSL_PKCS12_get_certificate and QSCOSSL_PKCS12_get_private_key return separately owned decoded results as documented by their headers. Check supported algorithms and password handling; the interface is for import, not PKCS#12 export.

14.4 CSR and lifecycle

QSCOSSL_CSR_verify_der/pem verifies the supplied certification request, including signature checks. This validates request integrity but does not authorize subject names, roles or certificate issuance. The QSCOSSL_CSR_validate_issuance_der preflight operation checks request acceptability under a bounded policy; it does not sign or issue a certificate. Conversion helpers QSCOSSL_CERT_to_pem, QSCOSSL_CSR_to_pem and QSCOSSL_CRL_to_pem produce caller-owned PEM output with explicit capacity checks.

14.5 CRL and offline OCSP

QSCOSSL_CRL_check_der accepts caller-owned DER for the CRL, issuer and subject, and an explicit 15-byte YYYYMMDDHHMMSSZ validation time. It returns REVOKED, NOT_REVOKED, or ERROR only after the necessary signed CRL and issuer checks. ERROR is never equivalent to NOT_REVOKED. The caller must validate issuer trust and certificate path separately.

QSCOSSL_OCSP_check_der accepts a locally supplied signed DER response with matching issuer/subject and evaluation time; its results are GOOD, REVOKED, UNKNOWN and ERROR. UNKNOWN is an authenticated negative knowledge result, not approval; ERROR includes parse, signature, identity and freshness failures. Neither function downloads revocation objects or establishes issuer trust independently.

14.6 Verification ordering and incident response

A robust certificate acceptance path begins with parsing and strict input bounds; it next builds a path to an independently trusted anchor and applies identity, validity-period and usage policy. Only after those prerequisites should a caller use an authenticated offline CRL/OCSP result to decide whether the certificate is acceptable.

An OCSP UNKNOWN value is not equivalent to GOOD; a CRL or OCSP ERROR value is not equivalent to a clean revocation result. The application must choose and document its failure policy for stale, absent or contradictory evidence. A successful CSR signature similarly proves request integrity, not permission to issue a certificate.

Test negative fixtures for wrong issuer, changed serial number, malformed DER, stale signed evidence, mismatched certificate identity and untrusted roots. Store enough diagnostic information to explain a rejection without exposing private keys, password contents or confidential application data.

15: TLS 1.3 Client and Server

15.1 Supported interface model

The SSL-style wrapper maps selected client and server methods, context policy, identities, native or QSC-managed sockets, TLS handshakes, authenticated I/O and shutdown onto the QSC TLS 1.3 implementation. TLS_client_method and TLS_server_method identify connection roles; no general legacy SSL/TLS protocol version compatibility is implied.

15.2 Client call hierarchy

```
OPENSSL_init_ssl -> SSL_CTX_new(TLS_client_method())
        -> SSL_CTX_load_verify_locations
        -> SSL_CTX_set_verify(SSL_VERIFY_PEER)
        -> SSL_new -> SSL_set_tlsext_host_name
        -> QSCOSSL_SSL_set_host_service / SSL_set_fd
        -> SSL_connect -> SSL_get_verify_result
        -> SSL_write / SSL_read -> SSL_shutdown
        -> SSL_free -> SSL_CTX_free
```

The network hostname must be verified against the authenticated peer identity; SNI alone selects a server-side identity and is not itself hostname verification. Check handshake result and verification policy before exchanging application data. The complete standalone example is examples/tls_client_example.c and expects hostname, service and CA bundle arguments.

15.3 Server call hierarchy

```
OPENSSL_init_ssl -> SSL_CTX_new(TLS_server_method())
        -> SSL_CTX_use_certificate_file
        -> SSL_CTX_use_PrivateKey_file
```

-> [TLS policy, client authentication, SNI identity]
-> QSCOSSL_SSL_CTX_bind / native socket
-> SSL_new -> SSL_accept
-> SSL_read / SSL_write -> SSL_shutdown
-> SSL_free -> SSL_CTX_free

The server must load a matching certificate and private key. Client-authentication trust anchors are separate from server identity. Configure supported TLS policy before bind or connection creation; the QSCOSSL_* interface includes cipher-suite IDs, named groups, signature schemes, ALPN, SNI identity choices, timeouts and CRL policy. The provided server example accepts address, port, certificate and private-key arguments.

15.4 Borrowed sockets and fail-closed validation

SSL_set_fd accepts an already connected blocking IPv4/IPv6 TCP socket and retains only a non-owning reference. Reject invalid, disconnected or nonblocking descriptors before creating TLS state. Windows native SOCKET values wider than int must use QSCOSSL_SSL_set_native_socket. SSL_shutdown and SSL_free close the TLS layer; they do not close the caller-owned TCP descriptor. A caller who owns the descriptor must still close it after TLS shutdown and release.

15.5 Peer certificate and application data

SSL_get_peer_certificate returns an independently owned authenticated leaf certificate only after the successful peer-authenticated handshake. SSL_get_verify_result reports verification status, but applications must still apply host/service policy. Treat SSL_read/SSL_write return values and SSL_get_error as operation status, not as guarantees of complete application-record transfer; implement framing, retries and partial-I/O management for the chosen blocking model.

15.6 TLS deployment and socket lifecycle

TLS configuration should be complete before the first handshake: selected role, accepted trust anchors, server identity and private key, permitted named groups and signature schemes, SNI/ALPN policy, and peer-verification expectations. Server-side client authentication requires its own trust material rather than reuse of the server certificate.

For an attached external TCP socket, QSCOSSL borrows the connected blocking descriptor. The caller owns creation, transport-level error handling and final socket

closure. Treat a failed SSL_connect or SSL_accept as a failed connection; free the SSL layer and resolve socket ownership without retrying the same compromised state blindly.

After a handshake, interpret SSL_read and SSL_write results as byte counts and status indications. The application protocol must frame complete messages and handle partial transfers and shutdown; a successful TLS record operation does not imply the remote application has received a complete higher-level request.

16: OpenSSL 3 Provider and Hardware Adapters

16.1 Separate provider construction

QSCOSSL_BUILD_OPENSSL3_PROVIDER=ON builds a genuine OpenSSL 3 provider module using the installed OpenSSL development package and QSC cryptographic archive. This build must not include QSCOSSL's replacement openssl/*.h headers. On ELF platforms, the static QSC archive must be position independent. The provider uses OpenSSL fetch/dispatch rather than QSCOSSL EVP_* compatibility objects.

16.2 Provider operation inventory

Operation	Implemented names / modes	Important limits
Digest	SHA256, SHA384, SHA512; SHA3-256, SHA3-512	Incremental fixed-output digest.
XOF	SHAKE128, SHAKE256	Incremental; max 16 MiB per output call.
MAC	HMAC-SHA2, KMAC128/256, Poly1305	KMAC fixed-output only; Poly1305 one-time key.
KDF	HKDF-SHA2 (extract + expand; extract-only; expand-only)	Extract-only requires digest-length output.
Password KDF	PBKDF2-HMAC-SHA2	Configurable input and work-factor bounds; no weak-policy assumptions.
Cipher	AES-128/256-CTR, AES-128/256-CBC, ChaCha20	No AEAD ciphers, no key-management or signature provider operations.

Table 16.1 Implemented OpenSSL 3 provider categories

16.3 Hardware-key integration

qscossl_hw.h provides optional is_available, key_sign, key_agree and key_attest entry points over the QSC hardware abstraction. Presence of the header is not proof of an attached HSM, TPM, PKCS#11 token or production key. Physical-device testing is

required to establish non-exportability, provider policy, device failure behavior and attestation correctness. Do not silently fall back to software operations if hardware policy requires an external key.

16.4 Provider deployment checks

The native OpenSSL 3 provider follows a separate build and load process using the genuine OpenSSL headers, shared-library loader and provider API. Its supported operations are limited to the categories in Table 16.1. It does not automatically provide every cipher, authenticated mode, key-management algorithm or TLS capability of the compatibility wrapper.

At startup, the hosting application should verify the provider module was actually loaded and the intended operation fetched successfully. Record its provider selection and accepted property query. A successful default OpenSSL fetch can accidentally select an unrelated provider if the deployment assumes all algorithms now use QSC.

The optional hardware adapter is a separate trust decision: a supported header or compile option cannot establish that an HSM/TPM/token is present, that keys remain non-exportable, or that attestation succeeded. Device-aware failure and audit testing are mandatory where a security policy requires hardware key custody.

17: Integration Examples and Migration Patterns

17.1 Recommended migration method

- Inventory all OpenSSL headers, symbols, configuration and binary dependencies in the target application.
- Classify every call as supported, QSCOSSL extension, optional provider-only or unsupported; do not rely on compile success alone.
- Pin the intended QSC source revision and cryptographic build profile, then configure wrapper and examples against it.
- Replace include paths with the QSCOSSL openssl headers; keep the provider integration as a different binary target.
- Adapt AEAD decrypt finalization to QSCOSSL_EVP_DecryptFinal_ex2 and verify caller output capacity.
- Review certificate trust, hostname checks, key serialization, nonce management, password policy and socket ownership.

- Run known-answer tests, negative cases, interaction tests, sanitizers, TLS interoperability and native target tests.
- Archive source hashes, compiler flags, test results, limitations and signed release artifacts for deployment.

17.2 Worked example inventory

Source example	What to study
evp_digest_example.c	EVP SHA3-512 allocation, update, final and cleanup.
evp_cipher_example.c	AES EVP block/stream cipher use and output length handling.
evp_aead_example.c	AES-256-GCM AAD, tag get/set and capacity-aware decrypt finalization.
hmac_mac_example.c	HMAC and QSC MAC keying and finalization.
rand_example.c	Configured QSC random provider and RAND_bytes use.
pqc_kem_sign_example.c	ML-KEM shared-secret exchange and ML-DSA detached sign/verify.
tls_client_example.c	QSC TLS client with CA trust, SNI and configured endpoint.
tls_server_example.c	TLS server identity, bind, accept and protected application I/O.
x509_verify_example.c	PEM certificate loading, trust store and host checking.

Table 17.1 Project implementation examples

These tutorial examples require their documented fixtures and peer configuration. Use only audited, application-specific key and certificate material in production.

17.3 Example-driven acceptance checklist

Use the supplied examples as starting points, then replace demonstration keys, file paths, certificate stores and peer addresses with deployment-controlled inputs. Exercise the success path and at least one failure path for every primitive exposed by the application; example output alone is not a qualification record.

For EVP workflows, assert exact processed lengths and test short output buffers, incorrect keys, changed tags, reordered AAD, invalid parameters and repeated initialization. For X.509, test unknown roots, wrong hostnames, expired certificates, untrusted intermediates, bad revocation signatures and an authenticated UNKNOWN OCSP status.

For TLS, separate local loopback smoke tests from multi-host interoperability and application-framing checks. Record whether each test was executed and which machine,

operating system, compiler profile and dependency revision were used; do not mark inherited project records as a new test run.

18: Testing, Security Review and Limitations

18.1 Current source-package evidence

The README dated November 8, 2026 records 22 offline Linux GCC stage executables passing after corrections to inherited Stage 4 and Stage 8 tests. The optional OpenSSL 3 provider has a separate test matrix. Dated engineering reports describe full/compact/sanitizer runs, optional loopback TLS and earlier inherited expectation conflicts. These are project-supplied historical records; no new QSCOSSL or QSC binaries were built or tests executed to prepare this paper.

The ZIP contains Windows Debug .obj/.lib and a test .exe, but their presence alone is not an independently validated current MSVC run. Native Windows, macOS and ARM64 release qualification, physical key devices and external compliance validation remain unverified in the documentation examined.

18.2 Security review areas

- AEAD: require unique nonces, authenticate AAD, check tags and capacities, and do not release plaintext before verified finalization.
- Ciphers: authenticate CBC/CTR records at the protocol layer; avoid ECB for general confidentiality; review buffer overlap.
- KDF: reject invalid lengths, separate salt/info, enforce password iteration work factors and wipe secrets.
- PQC: select correct parameter sets and public-key policies; distinguish implicit rejection from ciphertext authenticity.
- Key ownership: match alloc/free, do not serialize opaque structs by memcpy and do not share mutable contexts.
- PKI: establish issuer trust, certificate path validation, hostname matching and explicit CRL/OCSP unknown/error policy.
- TLS: configure supported protocol policy, SNI/ALPN, client auth and connected blocking socket ownership correctly.
- Provider: test genuine OpenSSL provider loading, fetch aliases, implementation failures and cross-build OpenSSL compatibility.
- Portability: qualify scalar and intended SIMD CPU profiles; do not execute unsupported instruction targets or rely on unrun tests.

18.3 Qualification evidence and deployment boundary

The supplied engineering reports contain historical Windows and Linux stage results as well as specific notes on the OpenSSL provider and certificate adapters. These notes should be interpreted with their dates, configurations and known failures. They are not substitutes for a qualification run using the final release archives.

An implementor should retain a test ledger recording function family, test identifier, expected output or failure, build profile, platform, outcome and evidence file. Repeat negative authentication, parsing, allocation and threading checks whenever a compiler profile or external QSC revision changes.

Neither the specification nor the presence of known-answer tests establishes formal FIPS 140 validation, OpenSSL ABI parity, certificate-path policy completeness, or deployment-level cryptographic security. Those assessments require separate scope definitions and independent evidence.

The following header-organized reference incorporates parameter and return annotations from the supplied source headers. Parameter documentation is intentionally omitted where the source offers only generic boilerplate. Buffer sizes, ownership and failure semantics that are particularly sensitive to misuse remain documented in the preceding chapters and in the current header text.

19: Public Header API Reference

This section contains all public function signatures bearing QSCOSSL_API in the 26 distributed public headers. Prototypes are reproduced from the uploaded source; the adjacent synopsis follows the closest Doxygen \brief when available. Conditional interfaces require their documented feature macros. This is a lookup reference, not a promise that every OpenSSL selector is implemented for every QSC build profile.

19.1 openssl / bio.h (4 declarations)

File and memory input streams.

BIO_free_all

void BIO_free_all(BIO* bio);

Release the BIO free all resources and associated owned state.

Parameters: bio: Pointer to caller-supplied data or state; see API ownership conventions.

Return: No value is returned.

BIO_new_file

BIO* BIO_new_file(const char* filename, const char* mode);

Allocate a new BIO new file handle or operation context.

Parameters: filename: NUL-terminated file path.; mode: Operation mode selector.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

BIO_new_mem_buf

BIO* BIO_new_mem_buf(const void* buffer, int length);

Allocate a new BIO new mem buf handle or operation context.

Parameters: buffer: Caller-provided byte buffer.; length: Buffer length in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

BIO_read

int BIO_read(BIO* bio, void* output, int length);

Read data from the selected source into the provided buffer.

Parameters: bio: Pointer to caller-supplied data or state; see API ownership conventions.; length: Buffer length in bytes.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

19.2 openssl / crypto.h (15 declarations)

Memory allocation and secure-memory primitives.

OPENSSL_cleanse

void OPENSSL_cleanse(void* pointer, size_t length);

Securely erase the requested memory range.

Parameters: pointer: Caller-provided pointer.; length: Buffer length in bytes.

Return: No value is returned.

OPENSSL_clear_free

void OPENSSL_clear_free(void* pointer, size_t length);

Release the OPENSSL clear free resources and associated owned state.

Parameters: pointer: Caller-provided pointer.; length: Buffer length in bytes.

Return: No value is returned.

OPENSSL_secure_clear_free

void OPENSSL_secure_clear_free(void* pointer, size_t length);

Release the OPENSSL secure clear free resources and associated owned state.

Parameters: pointer: Caller-provided pointer.; length: Buffer length in bytes.

Return: No value is returned.

OPENSSL_free

void OPENSSL_free(void* pointer);

Release the OPENSSL free resources and associated owned state.

Parameters: pointer: Caller-provided pointer.

Return: No value is returned.

OPENSSL_malloc

void* OPENSSL_malloc(size_t length);

Allocate the requested number of bytes.

Parameters: length: Buffer length in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

OPENSSL_realloc

void* OPENSSL_realloc(void* pointer, size_t length);

Resize the owned allocation.

Parameters: pointer: Caller-provided pointer.; length: Buffer length in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

OPENSSL_zalloc

void* OPENSSL_zalloc(size_t length);

Allocate and zero-initialize the requested heap byte count; release with OPENSSL_free.

Parameters: length: Buffer length in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

OPENSSL_strdup

char* OPENSSL_strdup(const char* source);

Duplicate a NUL-terminated string into a new, caller-owned allocation.

Parameters: source: Source data or object.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

OPENSSL_init_crypto

int OPENSSL_init_crypto(uint64_t options, const void* settings);

Initialize the QSCOSSL compatibility core and report initialization status.

Parameters: options: Configuration flags.; settings: Optional configuration structure.

OpenSSL_version

const char* OpenSSL_version(int type);

Return QSCOSSL version/compatibility identification text.

Parameters: type: Type selector.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

OpenSSL_version_num

unsigned long OpenSSL_version_num(void);

Return the QSCOSSL compatibility numeric version, not the actual OpenSSL runtime.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

CRYPTO_free

void CRYPTO_free(void* pointer, const char* file, int line);

Release the CRYPTO free resources and associated owned state.

Parameters: pointer: Caller-provided pointer.; file: File path or source identifier.; line: Value used to configure or select the operation.

Return: No value is returned.

CRYPTO_malloc

void* CRYPTO_malloc(size_t length, const char* file, int line);

Allocate the requested number of bytes.

Parameters: length: Buffer length in bytes.; file: File path or source identifier.; line: Value used to configure or select the operation.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

CRYPTO_realloc

void* CRYPTO_realloc(void* pointer, size_t length, const char* file, int line);

Resize the owned allocation.

Parameters: pointer: Caller-provided pointer.; length: Buffer length in bytes.; file: File path or source identifier.; line: Value used to configure or select the operation.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

CRYPTO_memcmp

int CRYPTO_memcmp(const void* left, const void* right, size_t length);

Compare buffers with the specified memory comparison semantics.

Parameters: left: Pointer to caller-supplied data or state; see API ownership conventions.; right: Pointer to caller-supplied data or state; see API ownership conventions.; length: Buffer length in bytes.

19.3 openssl / err.h (6 declarations)

Error diagnostics and handling.

ERR_clear_error

void ERR_clear_error(void);

Clear the current wrapper error state.

Return: No value is returned.

ERR_get_error

unsigned long ERR_get_error(void);

Consume and return the recorded wrapper error code.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

ERR_peek_error

unsigned long ERR_peek_error(void);

Inspect the current wrapper error code without consuming it.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

ERR_reason_error_string

const char* ERR_reason_error_string(unsigned long errorcode);

Return the borrowed diagnostic reason for an error code.

Parameters: errorcode: Value used to configure or select the operation.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

ERR_error_string_n

void ERR_error_string_n(unsigned long errorcode, char* buffer, size_t length);

Format the selected error code into a bounded caller buffer.

Parameters: errorcode: Value used to configure or select the operation.; buffer: Caller-provided byte buffer.; length: Buffer length in bytes.

Return: No value is returned.

ERR_put_error

void ERR_put_error(int library, int functioncode, int reason, const char* file, int line);

Record an error using OpenSSL-style library/function/reason information.

Parameters: library: Value used to configure or select the operation.; functioncode: Value used to configure or select the operation.; reason: Value used to configure or select the operation.; file: File path or source identifier.; line: Value used to configure or select the operation.

Return: No value is returned.

19.4 openssl / evp.h (68 declarations)

EVP symmetric encryption, hashing, and asymmetric operations.

EVP_CIPHER_CTX_new

EVP_CIPHER_CTX* EVP_CIPHER_CTX_new(void);

Allocate a new, caller-owned EVP cipher context. Check for NULL and release with EVP_CIPHER_CTX_free.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

EVP_CIPHER_CTX_reset

int EVP_CIPHER_CTX_reset(EVP_CIPHER_CTX* context);

Clear the state in an existing cipher context so it can be configured for a subsequent operation; it remains allocated.

EVP_CIPHER_CTX_free

void EVP_CIPHER_CTX_free(EVP_CIPHER_CTX* context);

Release the cipher context and its owned operational state. Do not release borrowed cipher descriptors.

Return: No value is returned.

EVP_CIPHER_CTX_ctrl

int EVP_CIPHER_CTX_ctrl(EVP_CIPHER_CTX* context, int control, int argument, void* pointer);

Apply a cipher-specific control such as AEAD tag retrieval, tag setting or IV size.

Parameters: control: Value used to configure or select the operation.; argument: Value used to configure or select the operation.; pointer: Caller-provided pointer.

EVP_CIPHER_CTX_set_padding

int EVP_CIPHER_CTX_set_padding(EVP_CIPHER_CTX* context, int padding);

Enable or disable EVP block-mode padding on the owned cipher context. Explicitly handle whole-block requirements when disabled.

Parameters: padding: Value used to configure or select the operation.

EVP_CIPHER_get_key_length

int EVP_CIPHER_get_key_length(const EVP_CIPHER* cipher);

Return the selected cipher descriptor's key length in bytes.

Parameters: cipher: Cipher descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_CIPHER_get_iv_length

int EVP_CIPHER_get_iv_length(const EVP_CIPHER* cipher);

Return the selected cipher descriptor's nonce/IV length in bytes.

Parameters: cipher: Cipher descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_CIPHER_get_block_size

int EVP_CIPHER_get_block_size(const EVP_CIPHER* cipher);

Return the selected cipher descriptor's block size in bytes.

Parameters: cipher: Cipher descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_CIPHER_get_mode

int EVP_CIPHER_get_mode(const EVP_CIPHER* cipher);

Return the selected cipher descriptor's algorithm mode identifier.

Parameters: cipher: Cipher descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_CIPHER_get0_name

const char* EVP_CIPHER_get0_name(const EVP_CIPHER* cipher);

Return the borrowed name of the selected cipher descriptor; do not free it.

Parameters: cipher: Cipher descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_CIPHER_key_length

int EVP_CIPHER_key_length(const EVP_CIPHER* cipher);

Query the key length in bytes using the compatibility spelling.

Parameters: cipher: Cipher descriptor.

Return: Requested length or size, or a failure indicator.

EVP_CIPHER_iv_length

int EVP_CIPHER_iv_length(const EVP_CIPHER* cipher);

Query the IV length in bytes using the compatibility spelling.

Parameters: cipher: Cipher descriptor.

Return: Requested length or size, or a failure indicator.

EVP_CIPHER_block_size

int EVP_CIPHER_block_size(const EVP_CIPHER* cipher);

Query the cipher block size in bytes using the compatibility spelling.

Parameters: cipher: Cipher descriptor.

Return: Requested length or size, or a failure indicator.

EVP_aes_128_cbc

const EVP_CIPHER* EVP_aes_128_cbc(void);

Return the aes-128-cbc algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_128_ecb

const EVP_CIPHER* EVP_aes_128_ecb(void);

Return the aes-128-ecb algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_128_ctr

const EVP_CIPHER* EVP_aes_128_ctr(void);

Return the aes-128-ctr algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_128_gcm

const EVP_CIPHER* EVP_aes_128_gcm(void);

Return the aes-128-gcm algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_256_cbc

const EVP_CIPHER* EVP_aes_256_cbc(void);

Return the aes-256-cbc algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_256_ecb

const EVP_CIPHER* EVP_aes_256_ecb(void);

Return the aes-256-ecb algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_256_ctr

const EVP_CIPHER* EVP_aes_256_ctr(void);

Return the aes-256-ctr algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_aes_256_gcm

const EVP_CIPHER* EVP_aes_256_gcm(void);

Return the aes-256-gcm algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_chacha20

const EVP_CIPHER* EVP_chacha20(void);

Return the chacha20 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_chacha20_poly1305

const EVP_CIPHER* EVP_chacha20_poly1305(void);

Return the chacha20-poly1305 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_DecryptFinal_ex

int EVP_DecryptFinal_ex(EVP_CIPHER_CTX* context, uint8_t* output, int* outputlength);

Finalize decryption and return any final output.

EVP_DecryptInit_ex

int EVP_DecryptInit_ex(EVP_CIPHER_CTX* context, const EVP_CIPHER* cipher, ENGINE* engine, const uint8_t* key, const uint8_t* iv);

Initialize the EVP context for decryption using the selected cipher and key.

Parameters: cipher: Cipher descriptor.; engine: Reserved engine parameter; see implementation support.; key: Key material or key handle.; iv: Initialization vector.

EVP_DecryptUpdate

int EVP_DecryptUpdate(EVP_CIPHER_CTX* context, uint8_t* output, int* outputlength, const uint8_t* input, int inputlength);

Process an incremental decryption input chunk.

EVP_EncryptFinal_ex

int EVP_EncryptFinal_ex(EVP_CIPHER_CTX* context, uint8_t* output, int* outputlength);

Finalize encryption and return any final output.

EVP_EncryptInit_ex

int EVP_EncryptInit_ex(EVP_CIPHER_CTX* context, const EVP_CIPHER* cipher, ENGINE* engine, const uint8_t* key, const uint8_t* iv);

Initialize the EVP context for encryption using the selected cipher and key.

Parameters: cipher: Cipher descriptor.; engine: Reserved engine parameter; see implementation support.; key: Key material or key handle.; iv: Initialization vector.

EVP_EncryptUpdate

int EVP_EncryptUpdate(EVP_CIPHER_CTX* context, uint8_t* output, int* outputlength, const uint8_t* input, int inputlength);

Process an incremental encryption input chunk.

EVP_MD_CTX_new

EVP_MD_CTX* EVP_MD_CTX_new(void);

Allocate a new EVP MD CTX new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

EVP_MD_CTX_reset

int EVP_MD_CTX_reset(EVP_MD_CTX* context);

Reset the EVP MD CTX reset state for reuse.

EVP_MD_CTX_free

void EVP_MD_CTX_free(EVP_MD_CTX* context);

Release the EVP MD CTX free resources and associated owned state.

Return: No value is returned.

EVP_MD_get_size

int EVP_MD_get_size(const EVP_MD* digest);

Return the digest output size in bytes for the selected hash descriptor.

Parameters: digest: Digest algorithm descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_MD_get_block_size

int EVP_MD_get_block_size(const EVP_MD* digest);

Return the message processing block size in bytes for the selected digest.

Parameters: digest: Digest algorithm descriptor.

Return: Requested property, or a failure indicator for invalid arguments.

EVP_MD_get0_name

const char* EVP_MD_get0_name(const EVP_MD* digest);

Return the borrowed algorithm name associated with the digest descriptor.

Parameters: digest: Digest algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_MD_size

int EVP_MD_size(const EVP_MD* digest);

Return the selected digest output length, in bytes.

Parameters: digest: Digest algorithm descriptor.

Return: Requested length or size, or a failure indicator.

EVP_MD_block_size

int EVP_MD_block_size(const EVP_MD* digest);

Return the selected digest input block size, in bytes.

Parameters: digest: Digest algorithm descriptor.

Return: Requested length or size, or a failure indicator.

EVP_sha256

const EVP_MD* EVP_sha256(void);

Return the sha256 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_sha384

const EVP_MD* EVP_sha384(void);

Return the sha384 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_sha512

const EVP_MD* EVP_sha512(void);

Return the sha512 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_sha3_256

const EVP_MD* EVP_sha3_256(void);

Return the sha3-256 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_sha3_512

const EVP_MD* EVP_sha3_512(void);

Return the sha3-512 algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

EVP_Digest

int EVP_Digest(const void* input, size_t inputlength, uint8_t* output, unsigned int* outputlength, const EVP_MD* digest, ENGINE* engine);

Compute or query the selected cryptographic digest.

Parameters: digest: Digest algorithm descriptor.; engine: Reserved engine parameter; see implementation support.

EVP_DigestFinal_ex

int EVP_DigestFinal_ex(EVP_MD_CTX* context, uint8_t* output, unsigned int* outputlength);

Finalize the active digest and write its output.

EVP_DigestInit_ex

int EVP_DigestInit_ex(EVP_MD_CTX* context, const EVP_MD* digest, ENGINE* engine);

Initialize the message-digest context with the selected hash.

Parameters: digest: Digest algorithm descriptor.; engine: Reserved engine parameter; see implementation support.

EVP_DigestUpdate

int EVP_DigestUpdate(EVP_MD_CTX* context, const void* input, size_t inputlength);

Absorb an input block into the active message digest.

PKCS5_PBKDF2_HMAC

int PKCS5_PBKDF2_HMAC(const char* password, int passwordlength, const uint8_t* salt, int saltlength, int iterations, const EVP_MD* digest, int keylength, uint8_t* output);

Derive a PBKDF2 key using a QSC SHA-2 HMAC implementation.

Parameters: password: Password bytes, NUL-terminated when passwordlength is -1.; passwordlength: Password byte length or -1 for text length.; salt: Salt bytes; NULL is permitted only when saltlength is zero.; saltlength: Non-negative salt byte length.; iterations: Positive PBKDF2 iteration count.; digest: SHA-256, SHA-384, or SHA-512.; keylength: Positive output byte count.; output: Caller-owned writable keylength-byte buffer.

Return: 1 on success; 0 on invalid parameters or a derivation failure.

Notes: Output is cleared on failure; no secret material is retained. Thread-safe.

EVP_PKEY_new

EVP_PKEY* EVP_PKEY_new(void);

Allocate a new EVP PKEY new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

EVP_PKEY_free

void EVP_PKEY_free(EVP_PKEY* key);

Release the EVP PKEY free resources and associated owned state.

Parameters: key: Key material or key handle.

Return: No value is returned.

EVP_PKEY_id

int EVP_PKEY_id(const EVP_PKEY* key);

Return the algorithm identifier of an EVP_PKEY object.

Parameters: key: Key material or key handle.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

EVP_PKEY_new_raw_private_key

EVP_PKEY* EVP_PKEY_new_raw_private_key(int type, ENGINE* engine, const uint8_t* key, size_t keylength);

Construct an owned EVP_PKEY from encoded raw private-key bytes for supported key types.

Parameters: type: Type selector.; engine: Reserved engine parameter; see implementation support.; key: Key material or key handle.; keylength: Key size in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

EVP_PKEY_new_raw_public_key

EVP_PKEY* EVP_PKEY_new_raw_public_key(int type, ENGINE* engine, const uint8_t* key, size_t keylength);

Construct an owned EVP_PKEY from raw public-key bytes for supported types.

Parameters: type: Type selector.; engine: Reserved engine parameter; see implementation support.; key: Key material or key handle.; keylength: Key size in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

EVP_PKEY_get_raw_private_key

int EVP_PKEY_get_raw_private_key(const EVP_PKEY* key, uint8_t* output, size_t* outputlength);

Export the raw private-key representation into caller-owned capacity and report length.

Parameters: key: Key material or key handle.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

EVP_PKEY_get_raw_public_key

int EVP_PKEY_get_raw_public_key(const EVP_PKEY* key, uint8_t* output, size_t* outputlength);

Export the raw public-key representation into caller-owned capacity and report length.

Parameters: key: Key material or key handle.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

EVP_PKEY_CTX_new

EVP_PKEY_CTX* EVP_PKEY_CTX_new(EVP_PKEY* key, ENGINE* engine);

Allocate a new EVP PKEY CTX new handle or operation context.

Parameters: key: Key material or key handle.; engine: Reserved engine parameter; see implementation support.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

EVP_PKEY_CTX_new_id

EVP_PKEY_CTX* EVP_PKEY_CTX_new_id(int id, ENGINE* engine);

Allocate a new EVP PKEY CTX new id handle or operation context.

Parameters: id: Value used to configure or select the operation.; engine: Reserved engine parameter; see implementation support.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

EVP_PKEY_CTX_free

void EVP_PKEY_CTX_free(EVP_PKEY_CTX* context);

Release the EVP PKEY CTX free resources and associated owned state.

Return: No value is returned.

EVP_PKEY_keygen_init

int EVP_PKEY_keygen_init(EVP_PKEY_CTX* context);

Initialize supported asymmetric key generation for a selected EVP_PKEY_CTX.

EVP_PKEY_keygen

int EVP_PKEY_keygen(EVP_PKEY_CTX* context, EVP_PKEY** key);

Generate an owned asymmetric key pair using the configured key-generation context.

Parameters: key: Key material or key handle.

EVP_PKEY_derive_init

int EVP_PKEY_derive_init(EVP_PKEY_CTX* context);

Initialize key agreement using the selected local private key context.

EVP_PKEY_derive_set_peer

int EVP_PKEY_derive_set_peer(EVP_PKEY_CTX* context, EVP_PKEY* peerkey);

Set the requested EVP PKEY derive peer property.

Parameters: peerkey: Pointer to caller-supplied data or state; see API ownership conventions.

EVP_PKEY_derive

int EVP_PKEY_derive(EVP_PKEY_CTX* context, uint8_t* output, size_t* outputlength);

Produce the shared secret or its required length from the local and peer keys.

EVP_DigestSignInit

int EVP_DigestSignInit(EVP_MD_CTX* context, EVP_PKEY_CTX** pkeycontext, const EVP_MD* digest, ENGINE* engine, EVP_PKEY* key);

Initialize QSC-backed signature generation with the selected private key.

Parameters: context: Mutable EVP_MD_CTX allocated by EVP_MD_CTX_new().; pkeycontext: Optional output pointer for the signing key context, owned by context; do not independently free it.; digest: Requested digest descriptor; pass NULL for intrinsic-hash schemes such as ML-DSA, EdDSA, Falcon and SPHINCS+.; engine: Reserved OpenSSL ENGINE argument; pass NULL.; key: Initialized, supported signing private key; remains caller-owned and must outlive the operation.

Return: 1 on initialization; 0 for unsupported digest/key combinations or invalid state.

EVP_DigestSignUpdate

int EVP_DigestSignUpdate(EVP_MD_CTX* context, const void* input, size_t inputlength);

Process additional input in an active operation.

EVP_DigestSignFinal

int EVP_DigestSignFinal(EVP_MD_CTX* context, uint8_t* signature, size_t* signaturelength);

Query or generate the detached signature for accumulated message bytes.

Parameters: context: Active initialized signing context with message supplied using EVP_DigestSignUpdate().; signature: Writable destination; NULL requests the maximum required capacity.; signaturelength: In/out byte count: capacity on entry, actual signature length on success, required capacity on short-buffer failure.

Return: 1 on successful query/signing, otherwise 0 with diagnostic error state.

EVP_DigestVerifyInit

int EVP_DigestVerifyInit(EVP_MD_CTX* context, EVP_PKEY_CTX** pkeycontext, const EVP_MD* digest, ENGINE* engine, EVP_PKEY* key);

Initialize verification for a matching public key and signature scheme.

Parameters: context: Mutable EVP_MD_CTX allocated by EVP_MD_CTX_new().; pkeycontext: Optional pointer receiving the context-owned EVP_PKEY_CTX.; digest: Scheme-compatible digest or NULL for ML-DSA and other intrinsic-hash signatures.; engine: Reserved OpenSSL ENGINE argument; pass NULL.; key: Initialized public/verification key; remains caller-owned during verification.

Return: 1 for a supported verification configuration; 0 on errors.

EVP_DigestVerifyUpdate

int EVP_DigestVerifyUpdate(EVP_MD_CTX* context, const void* input, size_t inputlength);

Process additional input in an active operation.

EVP_DigestVerifyFinal

int EVP_DigestVerifyFinal(EVP_MD_CTX* context, const uint8_t* signature, size_t signaturelength);

Check the supplied detached signature over all accumulated message input.

Parameters: signature: Pointer to caller-supplied data or state; see API ownership conventions.; signaturelength: Size of signature in bytes.

19.5 openssl / hmac.h (7 declarations)

Keyed message authentication.

HMAC_CTX_new

HMAC_CTX* HMAC_CTX_new(void);

Allocate a new HMAC CTX new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

HMAC_CTX_free

void HMAC_CTX_free(HMAC_CTX* context);

Release the HMAC CTX free resources and associated owned state.

Return: No value is returned.

HMAC_CTX_reset

int HMAC_CTX_reset(HMAC_CTX* context);

Reset the HMAC CTX reset state for reuse.

HMAC_Init_ex

int HMAC_Init_ex(HMAC_CTX* context, const void* key, int keylength, const EVP_MD* digest, ENGINE* engine);

Initialize or configure HMAC with the supplied key and supported digest.

Parameters: key: Key material or key handle.; keylength: Key size in bytes.; digest: Digest algorithm descriptor.; engine: Reserved engine parameter; see implementation support.

HMAC_Update

int HMAC_Update(HMAC_CTX* context, const uint8_t* input, size_t inputlength);

Process additional input in an active operation.

HMAC_Final

int HMAC_Final(HMAC_CTX* context, uint8_t* output, unsigned int* outputlength);

Finalize the accumulated HMAC and report the tag byte length.

HMAC

uint8_t* HMAC(const EVP_MD* digest, const void* key, int keylength, const uint8_t* input, size_t inputlength, uint8_t* output, unsigned int* outputlength);

Compute HMAC for the specified key, digest and complete message in one operation.

Parameters: digest: Digest algorithm descriptor.; key: Key material or key handle.; keylength: Key size in bytes.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

19.6 openssl / kdf.h (0 declarations)

KDF compatibility selectors.

This public header defines compile-time selectors or compatibility types rather than exported function declarations. Consult Annex A and the current header for its macros and constants.

19.7 openssl / pem.h (4 declarations)

PEM parsing and private-key imports.

PEM_read_bio_X509

X509* PEM_read_bio_X509(BIO* bio, X509** certificate, void* callback, void* userdata);

Read data from the selected source into the provided buffer.

Parameters: bio: Pointer to caller-supplied data or state; see API ownership conventions.; certificate: Certificate object.; callback: Optional callback parameter; consult API compatibility notes.; userdata: Caller-supplied callback context.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

PEM_read_X509

X509* PEM_read_X509(void* file, X509** certificate, void* callback, void* userdata);

Read data from the selected source into the provided buffer.

Parameters: file: File path or source identifier.; certificate: Certificate object.; callback: Optional callback parameter; consult API compatibility notes.; userdata: Caller-supplied callback context.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

PEM_read_bio_PrivateKey

EVP_PKEY* PEM_read_bio_PrivateKey(BIO* bio, EVP_PKEY** key, void* callback, void* userdata);

Read SEC1 or PKCS#8 private key from a memory BIO, including QSC-supported PBES2 encryption.

Parameters: bio: Caller-owned readable memory BIO.; key: Optional pointer replaced only on success; caller frees returned EVP_PKEY.; callback: Must be NULL; OpenSSL passphrase callbacks are not implemented.; userdata: For encrypted PEM, optional pointer to a caller-owned NUL-terminated passphrase.

Return: Newly owned EVP_PKEY or NULL on error, bad password or unsupported encryption.

Notes: Supports PBES2/PBKDF2 with AES-128/256-CBC and QSC-supported HMAC-SHA2 PRFs only. Thread safe with distinct BIO and key objects. Do not reuse plaintext password memory after its lifetime.

PEM_read_PrivateKey

EVP_PKEY* PEM_read_PrivateKey(void* file, EVP_PKEY** key, void* callback, void* userdata);

Read SEC1 or PKCS#8 private key from an open file stream.

Parameters: file: Caller-owned readable FILE pointer; remains open on return.; key: Optional destination slot, replaced only on success.; callback: Must be NULL; passphrase callback integration is unsupported.; userdata: Optional NUL-terminated password for encrypted PEM; caller owned.

Return: Caller-owned key or NULL on error, bad password or unsupported encryption.

Notes: Same PBES2/PBKDF2 limitations and threading contract as PEM_read_bio_PrivateKey.

19.8 openssl / rand.h (6 declarations)

Cryptographic random generation.

QSCOSSL_RAND_set_provider

int QSCOSSL_RAND_set_provider(qscossl_random_provider provider);

Set the requested QSCOSSL RAND provider property.

Parameters: provider: Value used to configure or select the operation.

QSCOSSL_RAND_get_provider

qscossl_random_provider QSCOSSL_RAND_get_provider(void);

Return the current CSP/ACP random source selection.

Return: Result value as defined by this compatibility API.

RAND_bytes

int RAND_bytes(uint8_t* output, int length);

Fill the caller-provided buffer with cryptographically strong random bytes.

Parameters: length: Buffer length in bytes.

RAND_poll

int RAND_poll(void);

Request entropy provider refresh or status through the selected QSC provider.

RAND_priv_bytes

int RAND_priv_bytes(uint8_t* output, int length);

Fill the caller-provided buffer with cryptographically strong random bytes.

Parameters: length: Buffer length in bytes.

RAND_status

int RAND_status(void);

Report the selected random provider readiness state.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

19.9 openssl / ssl.h (41 declarations)

TLS 1.3 connection setup and QSC policy configuration.

OPENSSL_init_ssl

int OPENSSL_init_ssl(uint64_t options, const void* settings);

Initialize QSCOSSL TLS runtime support.

Parameters: options: Configuration flags.; settings: Optional configuration structure.

SSL_library_init

int SSL_library_init(void);

Compatibility TLS initialization entry point.

TLS_client_method

const SSL_METHOD* TLS_client_method(void);

Return the TLS-client-method algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

TLS_server_method

const SSL_METHOD* TLS_server_method(void);

Return the TLS-server-method algorithm descriptor.

Return: Pointer to the requested object or descriptor, or NULL if unavailable; ownership follows the named API.

SSL_CTX_new

SSL_CTX* SSL_CTX_new(const SSL_METHOD* method);

Allocate a new SSL CTX new handle or operation context.

Parameters: method: TLS method descriptor.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

SSL_CTX_free

void SSL_CTX_free(SSL_CTX* context);

Release the SSL CTX free resources and associated owned state.

Return: No value is returned.

SSL_CTX_use_certificate_file

int SSL_CTX_use_certificate_file(SSL_CTX* context, const char* file, int type);

Load the identity certificate or chain from a PEM/DER file for TLS.

Parameters: file: File path or source identifier.; type: Type selector.

SSL_CTX_use_PrivateKey_file

int SSL_CTX_use_PrivateKey_file(SSL_CTX* context, const char* file, int type);

Load the corresponding private-key material for the TLS identity.

Parameters: file: File path or source identifier.; type: Type selector.

SSL_CTX_load_verify_locations

int SSL_CTX_load_verify_locations(SSL_CTX* context, const char* cafile, const char* capath);

Verify the digital signature or certificate result.

Parameters: cafile: CA certificate file path.; capath: CA directory path.

SSL_CTX_set_verify

void SSL_CTX_set_verify(SSL_CTX* context, int mode, void* callback);

Configure the SSL CTX property.

Parameters: mode: Operation mode selector.; callback: Optional callback parameter; consult API compatibility notes.

Return: No value is returned.

SSL_CTX_set_min_proto_version

int SSL_CTX_set_min_proto_version(SSL_CTX* context, int version);

Configure the SSL CTX property.

Parameters: version: Value used to configure or select the operation.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

SSL_CTX_set_max_proto_version

int SSL_CTX_set_max_proto_version(SSL_CTX* context, int version);

Configure the SSL CTX property.

Parameters: version: Value used to configure or select the operation.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

SSL_CTX_set_alpn_protos

int SSL_CTX_set_alpn_protos(SSL_CTX* context, const unsigned char* protocols, unsigned int protocolslen);

Configure the SSL CTX property.

Parameters: protocols: Encoded protocol names.; protocolslen: Size of protocols in bytes.

SSL_new

SSL* SSL_new(SSL_CTX* context);

Allocate a new SSL new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

SSL_free

void SSL_free(SSL* ssl);

Release the SSL free resources and associated owned state.

Parameters: ssl: TLS connection handle.

Return: No value is returned.

SSL_set_fd

int SSL_set_fd(SSL* ssl, int filedescriptor);

Attach an already-connected blocking TCP socket without taking ownership.

Parameters: ssl: Valid SSL connection before handshake.; filedescriptor: Nonnegative native socket descriptor representable as int; Windows handles wider than int require QSCOSSL_SSL_set_native_socket().

Return: 1 if the socket is validated and attached; otherwise 0 and SSL_ERROR_SSL.

QSCOSSL_SSL_set_native_socket

int QSCOSSL_SSL_set_native_socket(SSL* ssl, uintptr_t nativehandle);

Register a full-width native socket without taking ownership.

Parameters: ssl: Unhandshaked QSCOSSL connection.; nativehandle: Connected operating-system TCP socket handle, passed as uintptr_t.

Return: 1 on validated registration, 0 on invalid socket/connection/state.

Notes: The caller retains socket ownership and must ensure blocking semantics. QSC may adjust socket timeout options during the handshake. Thread safety requires exclusive socket access.

SSL_set_tlsext_host_name

int SSL_set_tlsext_host_name(SSL* ssl, const char* name);

Set the requested SSL tlsext host name property.

Parameters: ssl: TLS connection handle.; name: NUL-terminated identifier string.

SSL_connect

int SSL_connect(SSL* ssl);

Perform a client-side authenticated TLS handshake.

Parameters: ssl: TLS connection handle.

SSL_accept

int SSL_accept(SSL* ssl);

Accept and authenticate an inbound TLS connection.

Parameters: ssl: TLS connection handle.

SSL_read

int SSL_read(SSL* ssl, void* buffer, int count);

Receive decrypted application bytes over an authenticated TLS session.

Parameters: ssl: Connection that has successfully completed SSL_connect() or SSL_accept().; buffer: Caller-owned writable output buffer with at least count bytes.; count: Positive maximum number of bytes to receive.

Return: Number of bytes read on success, or -1 on closed/error paths in this adapter.

SSL_write

int SSL_write(SSL* ssl, const void* buffer, int count);

Send application bytes over an established TLS session.

Parameters: ssl: Handshaked connection with appropriate peer authentication.; buffer: Caller-owned buffer holding at least count plaintext bytes.; count: Positive number of bytes requested for transmission.

Return: Number of bytes accepted for writing on success, or -1 on error.

SSL_get_error

int SSL_get_error(const SSL* ssl, int retcode);

Retrieve the last recorded TLS error from the supplied SSL object.

Parameters: ssl: Connection whose most recent TLS operation set the error.; retcode: Return code of the preceding SSL call; accepted for compatibility but not currently interpreted by this adapter.

Return: SSL_ERROR_NONE, SSL_ERROR_ZERO_RETURN, or SSL_ERROR_SSL as recorded by the connection; invalid objects yield SSL_ERROR_SSL.

SSL_shutdown

int SSL_shutdown(SSL* ssl);

Shut down the selected TLS connection without violating socket ownership.

Parameters: ssl: TLS connection handle.

SSL_get_verify_result

long SSL_get_verify_result(const SSL* ssl);

Return the current TLS certificate-verification status.

Parameters: ssl: TLS connection handle.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

SSL_get_peer_certificate

X509* SSL_get_peer_certificate(const SSL* ssl);

Obtain a new X509 object containing the authenticated peer leaf certificate.

Parameters: ssl: Caller-owned connection with a completed TLS handshake.

Return: A separately owned X509 object for a certificate-authenticated TLS session, otherwise NULL.

Notes: Caller must free a returned certificate using X509_free(). Server-side client-certificate sessions without an authenticated certificate return NULL.

QSCOSSL_SSL_set_host_service

int QSCOSSL_SSL_set_host_service(SSL* ssl, const char* hostname, const char* service);

Configure a QSC-managed TLS connection destination.

Parameters: ssl: Initialized TLS connection; retains its own copy of destination details.; hostname: Caller-owned NUL-terminated destination host name.; service: Caller-owned NUL-terminated port or service name.

Return: 1 when the host and service are configured, or 0 for invalid state or arguments.

Notes: This is an extension; the caller retains both input strings.

QSCOSSL_SSL_CTX_bind

int QSCOSSL_SSL_CTX_bind(SSL_CTX* context, const char* address, uint16_t port);

Bind a QSC-managed TLS server listener to the specified address and service.

Parameters: address: Pointer to caller-supplied data or state; see API ownership conventions.; port: TCP port number.

QSCOSSL_SSL_CTX_bind_ex

int QSCOSSL_SSL_CTX_bind_ex(SSL_CTX* context, const char* address, uint16_t port, int family);

Bind a QSC-owned TLS listener to a specified IPv4 or IPv6 address.

Parameters: context: Caller-owned TLS server context, idle and initialized.; address: Caller-owned NUL-terminated literal address of selected family.; port: Listener port (0 for automatic OS selection only if QSC permits it).; family: QSCOSSL_SOCKET_IPV4 or QSCOSSL_SOCKET_IPV6.

Return: 1 if bound, 0 if arguments or QSC network binding fail.

Notes: The listener owns its native socket; it is closed when SSL_CTX_free is called.

QSCOSSL_SSL_CTX_set_qsc_tls_policy

int QSCOSSL_SSL_CTX_set_qsc_tls_policy(SSL_CTX* context, int policy);

Configure the QSC TLS policy before binding a listener or opening connections.

Parameters: context: Initialized wrapper TLS context, caller-owned.; policy: One of QSCOSSL_TLS_POLICY_*; invalid values fail closed.

Return: 1 on success, 0 for unsupported policy or QSC configuration failure.

Notes: Do not mutate contexts concurrently with active TLS operations.

QSCOSSL_SSL_CTX_set_cipher_suites

int QSCOSSL_SSL_CTX_set_cipher_suites(SSL_CTX* context, const uint16_t* suites, size_t count);

Select TLS 1.3 cipher suites using their IANA numeric identifiers.

Parameters: context: Caller-owned, idle TLS context.; suites: Ordered caller-owned array of cipher suite identifiers.; count: Number of entries, from 1 through QSC_TLS_SOCKET_CIPHER_SUITE_MAX.

Return: 1 on success; 0 for unsupported identifiers, invalid state, or QSC errors.

Notes: Values are copied by QSC. Do not mutate a context with active connections.

QSCOSSL_SSL_CTX_set_named_groups

int QSCOSSL_SSL_CTX_set_named_groups(SSL_CTX* context, const uint16_t* groups, size_t count);

Set an ordered list of TLS key-exchange named-group identifiers.

Parameters: context: Caller-owned, idle TLS context.; groups: Ordered caller-owned array of IANA named-group identifiers.; count: Number of entries, from 1 through QSC_TLS_SOCKET_GROUP_MAX.

Return: 1 on success; 0 on rejected input or a QSC failure.

Notes: Includes explicitly named QSC PQC and hybrid groups; peer support is required.

QSCOSSL_SSL_CTX_set_signature_schemes

int QSCOSSL_SSL_CTX_set_signature_schemes(SSL_CTX* context, const uint16_t* schemes, size_t count);

Set TLS CertificateVerify signature scheme preferences.

Parameters: context: Caller-owned, idle TLS context.; schemes: Ordered caller-owned array of IANA/QSC signature identifiers.; count: Number of entries, from 1 through QSC_TLS_SOCKET_SIGNATURE_SCHEME_MAX.

Return: 1 on success; 0 on unsupported identifiers or QSC errors.

QSCOSSL_SSL_CTX_set_client_auth

int QSCOSSL_SSL_CTX_set_client_auth(SSL_CTX* context, int requestclientauth, int requireclientauth);

Configure QSC TLS server mutual-certificate-authentication policy.

Parameters: context: Caller-owned server TLS context, not yet bound.; requestclientauth: Nonzero requests a client certificate.; requireclientauth: Nonzero rejects clients without a valid certificate.

Return: 1 on success, 0 for unsupported modes, state, or QSC errors.

QSCOSSL_SSL_CTX_load_client_identity

int QSCOSSL_SSL_CTX_load_client_identity(SSL_CTX* context, const char* certificatepath, const char* privatekeypath, uint16_t scheme);

Load a client certificate and unencrypted private key for mutual TLS.

Parameters: context: Caller-owned client TLS context, not yet bound.; certificatepath: Path to the certificate chain file.; privatekeypath: Path to the associated private key file.; scheme: CertificateVerify signature identifier supported by QSC.

Return: 1 on success; 0 on failed loading, invalid arguments, or TLS errors.

QSCOSSL_SSL_CTX_add_server_identity

int QSCOSSL_SSL_CTX_add_server_identity(SSL_CTX* context, const char* hostname, const char* certificatepath, const char* privatekeypath, uint16_t scheme);

Register a named SNI certificate/key pair for a TLS server context.

Parameters: context: Caller-owned unbound TLS server context.; hostname: Hostname to bind to this identity.; certificatepath: Certificate chain file path.; privatekeypath: Private key file path.; scheme: CertificateVerify signature identifier supported by QSC.

Return: 1 on success; 0 on validation, load or TLS failure.

QSCOSSL_SSL_CTX_set_sni_required

int QSCOSSL_SSL_CTX_set_sni_required(SSL_CTX* context, int required);

Require SNI matching for a server context.

Parameters: context: Caller-owned unbound TLS server context.; required: Nonzero rejects missing or unmatched SNI hostnames.

Return: 1 on success; 0 if context is ineligible or QSC rejects the policy.

QSCOSSL_SSL_CTX_load_crl_file

int QSCOSSL_SSL_CTX_load_crl_file(SSL_CTX* context, const char* path);

Load a certificate revocation list into a QSC TLS context.

Parameters: context: Caller-owned unbound TLS context.; path: File path to a QSC-supported CRL.

Return: 1 on success; 0 on invalid arguments or verification configuration error.

QSCOSSL_SSL_CTX_set_timeouts

int QSCOSSL_SSL_CTX_set_timeouts(SSL_CTX* context, uint32_t handshakems, uint32_t receivems, uint32_t sendms);

Set blocking TLS socket and handshake timeout values.

Parameters: context: Caller-owned unbound TLS context.; handshakems: Cumulative handshake timeout in milliseconds; zero disables deadline.; receivems: Socket receive timeout in milliseconds.; sendms: Socket send timeout in milliseconds.

Return: 1 on success; 0 when QSC rejects the configuration.

QSCOSSL_SSL_CTX_set_qsc_signature_scheme

int QSCOSSL_SSL_CTX_set_qsc_signature_scheme(SSL_CTX* context, int scheme);

Configure the QSCOSSL SSL CTX property.

Parameters: scheme: Signature scheme identifier.

QSCOSSL_SSL_get_selected_alpn

int QSCOSSL_SSL_get_selected_alpn(SSL* ssl, char* protocol, size_t protocolcap, size_t* protocollen);

Obtain the selected ALPN identifier following negotiation.

Parameters: ssl: TLS connection handle.; protocol: Protocol identifier or buffer.; protocolcap: Available writable buffer capacity in bytes.; protocollen: Size of protocol in bytes.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

19.10 openssl / x509.h (18 declarations)

Certificate parsing, trust storage and verification.

X509_new

X509* X509_new(void);

Allocate a new X509 new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

X509_free

void X509_free(X509* certificate);

Release the X509 free resources and associated owned state.

Parameters: certificate: Certificate object.

Return: No value is returned.

d2i_X509

X509* d2i_X509(X509** certificate, const uint8_t** input, long length);

Decode a complete DER certificate into a QSCOSSL X509 object.

Parameters: certificate: Optional pointer to a reusable X509; when NULL or pointing to NULL a new object is created.; input: Address of the DER input pointer, which advances by the supplied length on success.; length: Exact number of DER bytes; must be positive.

Return: The decoded X509, or NULL on invalid input, allocation or parsing failure.

i2d_X509

int i2d_X509(X509* certificate, uint8_t** output);

Encode an existing certificate into DER and report the exact byte length.

Parameters: certificate: Previously decoded X509 to serialize.; output: Optional address of destination pointer. If NULL, only the required size is returned. If *output is NULL, QSCOSSL allocates the bytes; otherwise *output must point to enough writable space.

Return: Positive DER byte count on success, or -1 on failure.

X509_check_host

int X509_check_host(X509* certificate, const char* name, size_t namelen, unsigned int flags, char** peername);

Compare an authenticated certificate identity with the expected hostname.

Parameters: certificate: Certificate object.; name: NUL-terminated identifier string.; namelen: Size of name in bytes.; flags: Configuration flag bits.; peername: Pointer to caller-supplied data or state; see API ownership conventions.

X509_verify_cert

int X509_verify_cert(X509_STORE_CTX* context);

Build and verify the certificate path against the context's trusted store.

Parameters: context: Initialized X509_STORE_CTX with target leaf, independently trusted anchors, and optional intermediate certificates.

Return: 1 when QSC path verification succeeds; 0 for validation, allocation, or input failures.

X509_STORE_new

X509_STORE* X509_STORE_new(void);

Allocate a new X509 STORE new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

X509_STORE_free

void X509_STORE_free(X509_STORE* store);

Release the X509 STORE free resources and associated owned state.

Parameters: store: Certificate trust store.

Return: No value is returned.

X509_STORE_add_cert

int X509_STORE_add_cert(X509_STORE* store, X509* certificate);

Add an independently trusted certificate into an X509 trust store by copy.

Parameters: store: Certificate trust store.; certificate: Certificate object.

X509_STORE_load_locations

int X509_STORE_load_locations(X509_STORE* store, const char* cafile, const char* capath);

Load configured trust anchor certificates from supported locations.

Parameters: store: Certificate trust store.; cafile: CA certificate file path.; capath: CA directory path.

X509_STORE_CTX_new

X509_STORE_CTX* X509_STORE_CTX_new(void);

Allocate a new X509 STORE CTX new handle or operation context.

Return: Newly allocated result on success, or NULL on failure; release with the corresponding free function.

X509_STORE_CTX_free

void X509_STORE_CTX_free(X509_STORE_CTX* context);

Release the X509 STORE CTX free resources and associated owned state.

Return: No value is returned.

X509_STORE_CTX_init

int X509_STORE_CTX_init(X509_STORE_CTX* context, X509_STORE* store, X509* certificate, void* chain);

Associate the verification store, subject certificate and chain context.

Parameters: store: Certificate trust store.; certificate: Certificate object.; chain: Certificate chain.

QSCOSSL_X509_STORE_CTX_add_intermediate

int QSCOSSL_X509_STORE_CTX_add_intermediate(X509_STORE_CTX* context, const X509* certificate);

Copy one untrusted intermediate certificate into a verification context.

Parameters: context: Initialized caller-owned X509_STORE_CTX.; certificate: Caller-owned parsed certificate, retained by its caller.

Return: 1 on success, 0 for malformed input, unsupported certificates, or full chain.

Notes: Certificate DER is deep-copied. The context owns the copy until reinitialization, explicit clearing or X509_STORE_CTX_free(). Do not mutate the context concurrently.

QSCOSSL_X509_STORE_CTX_clear_intermediates

void QSCOSSL_X509_STORE_CTX_clear_intermediates(X509_STORE_CTX* context);

Clear intermediate certificates without modifying the verification leaf or store.

Parameters: context: Initialized caller-owned verification context.

Notes: Clears and releases all owned intermediate DER buffers; not thread-safe with concurrent use.

QSCOSSL_X509_STORE_CTX_intermediate_count

size_t QSCOSSL_X509_STORE_CTX_intermediate_count(const X509_STORE_CTX* context);

Read the number of intermediate certificates retained in a verification context.

Parameters: context: Initialized caller-owned verification context.

Return: Number of certificates or zero for invalid objects; caller retains the context.

Notes: Read-only; safe only when the context is not concurrently modified.

X509_STORE_CTX_get_error

long X509_STORE_CTX_get_error(const X509_STORE_CTX* context);

Retrieve the selected error property.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

X509_STORE_CTX_set_error

void X509_STORE_CTX_set_error(X509_STORE_CTX* context, int errorcode);

Configure the X509 STORE CTX property.

Parameters: errorcode: Value used to configure or select the operation.

Return: No value is returned.

19.11 qscossl.h (4 declarations)

Core QSCOSSL initialization and extension API.

qscossl_cleanup

int qscossl_cleanup(void);

Release QSCOSSL compatibility services.

qscossl_initialize

int qscossl_initialize(void);

Initialize QSCOSSL compatibility services.

qscossl_version_number

unsigned long qscossl_version_number(void);

Return the QSCOSSL compatibility version number.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

qscossl_version_text

const char* qscossl_version_text(void);

Return a read-only QSCOSSL compatibility version string.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

19.12 qscossl_aead.h (1 declarations)

Capacity-aware AEAD decryption.

QSCOSSL_EVP_DecryptFinal_ex2

int QSCOSSL_EVP_DecryptFinal_ex2(EVP_CIPHER_CTX* context, uint8_t* output, size_t outputcapacity, int* outputlength);

Finalize a buffered authenticated decryption with explicit output capacity.

Parameters: context: Initialized AEAD decryption context, exclusively owned by caller.; output: Writable plaintext buffer of outputcapacity bytes.; outputcapacity: Byte capacity of output; must accommodate all buffered ciphertext.; outputlength: Receives authenticated plaintext byte count or zero on failure.

Return: 1 on success, otherwise 0 and a QSCOSSL error is reported.

Warning: EVP_DecryptFinal_ex intentionally rejects buffered AEAD output because OpenSSL's signature cannot prove the caller supplied enough storage.

Notes: Authenticated plaintext is released only following successful tag verification. Contexts are not thread-safe. This operation does not transfer ownership.

19.13 qscossl_cert_lifecycle.h (4 declarations)

Certificate DER-to-PEM serialization and CSR preflight.

QSCOSSL_CERT_to_pem

int QSCOSSL_CERT_to_pem(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Convert a parsed X.509 certificate from DER to canonical PEM.

Parameters: der: Caller-owned complete DER certificate; never retained.; derlen: Number of DER bytes, from 1 to 65536.; pem: Optional caller-owned output; NULL queries the required capacity.; pemlen: On entry available bytes, on return required or actual bytes including NUL.

Return: One on successful conversion, zero on invalid DER or inadequate output space.

Notes: A syntactically valid certificate is not necessarily trusted. Independent calls are thread-safe.

QSCOSSL_CSR_to_pem

int QSCOSSL_CSR_to_pem(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Convert a signature-verified PKCS#10 CSR from DER to PEM.

Parameters: der: Caller-owned complete DER CSR; never retained.; derlen: Nonzero DER length, at most 16384 bytes.; pem: Optional caller-owned output; NULL queries required capacity.; pemlen: In/out capacity and result length including NUL.

Return: One on successful encoding, zero on invalid signature, DER, or capacity.

Notes: Signature validation does not authorize requested names or extensions. Thread-safe per call.

QSCOSSL_CRL_to_pem

int QSCOSSL_CRL_to_pem(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Convert a structurally valid CRL from DER to PEM without asserting trust.

Parameters: der: Caller-owned complete DER CRL, never retained.; derlen: Nonzero input length, at most 1048576 bytes.; pem: Optional caller-owned output, NULL for required capacity query.; pemlen: In/out capacity and result length including NUL.

Return: One on successful encoding, zero on failure.

Notes: Authentication, issuer and date checks must use QSCOSSL_CRL_check_der. Thread-safe per call.

QSCOSSL_CSR_validate_issuance_der

int QSCOSSL_CSR_validate_issuance_der(const uint8_t* der, size_t derlen);

Perform cryptographic and structural CSR preflight before certificate issuance.

Parameters: der: Caller-owned complete PKCS#10 DER input, never retained.; derlen: Nonzero input length, at most 16384 bytes.

Return: One only when CSR parsing, signature and QSC issuance structure validation succeed.

Notes: This is not certificate signing, CA authorization, identity validation or approval of requested extensions. Each invocation owns its temporary state and is thread-safe.

19.14 qscossl_config.h (0 declarations)

Platform and compile-time compatibility settings.

This public header defines compile-time selectors or compatibility types rather than exported function declarations. Consult Annex A and the current header for its macros and constants.

19.15 qscossl_crl.h (1 declarations)

Signed certificate revocation list checks.

QSCOSSL_CRL_check_der

qscossl_crl_result QSCOSSL_CRL_check_der(const uint8_t* crlder, size_t crllen, const uint8_t* issuerder, size_t issuerlen, const uint8_t* subjectder, size_t subjectlen, const char* utcgeneralizedtime, size_t timelen);

Check one certificate against a cryptographically verified CRL at an explicit time.

Parameters: crlder: Caller-owned DER CRL (up to 1 MiB); never retained.; crllen: CRL DER byte count; nonzero.; issuerder: Caller-owned DER certificate from an independently authenticated trusted issuer.; issuerlen: Issuer DER length; 1 to 65536 bytes.; subjectder: Caller-owned DER certificate whose revocation status is requested.; subjectlen: Subject DER length; 1 to 65536 bytes.; utcgeneralizedtime: Exactly 15 ASCII bytes in YYYYMMDDHHMMSSZ form, caller-owned.; timelen: Exactly 15 bytes, excluding any trailing NUL.

Return: qscossl_crl_result_revoked, qscossl_crl_result_not_revoked or qscossl_crl_result_error if any parsing, issuer, certificate signature, CRL signature, validity or revocation checking prerequisite fails.

Notes: Does NOT establish issuer trust or validate the entire certification chain. Authenticate issuerder independently before calling. Callers must use a trusted clock and decide how to handle unkno...

19.16 qscossl_csr.h (2 declarations)

PKCS#10 signed request verification.

QSCOSSL_CSR_verify_der

int QSCOSSL_CSR_verify_der(const uint8_t* der, size_t derlength);

Decode and verify a signed PKCS #10 certification request in DER form.

Parameters: der: Caller-owned byte array; never retained.; derlength: Nonzero DER byte count, at most 16384 bytes.

Return: 1 only when strict ASN.1 decoding and the CSR's signature verify.

Notes: A valid signature does not establish that an applicant is trustworthy. Independent calls are thread-safe.

QSCOSSL_CSR_verify_pem

int QSCOSSL_CSR_verify_pem(const char* pem, size_t pemlength);

Decode and verify a signed PKCS #10 CSR in PEM form.

Parameters: pem: Caller-owned PEM text buffer, not required to be NUL-terminated.; pemlength: Nonzero PEM length in bytes, at most 32768 bytes.

Return: 1 only on a successfully decoded, signature-verified CSR.

Notes: Signature verification is not CA policy approval or identity validation.

19.17 qscossl_errors.h (7 declarations)

Wrapper error codes and lookup functions.

qscossl_error_clear

void qscossl_error_clear(void);

Clear the QSCOSSL error record.

Return: No value is returned.

qscossl_error_get

unsigned long qscossl_error_get(void);

Consume the current QSCOSSL error code.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

qscossl_error_peek

unsigned long qscossl_error_peek(void);

Return the current QSCOSSL error code without consuming it.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

qscossl_error_reason_string

const char* qscossl_error_reason_string(unsigned long errorcode);

Return the human-readable reason for a QSCOSSL code.

Parameters: errorcode: Value used to configure or select the operation.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

qscossl_error_last_function

const char* qscossl_error_last_function(void);

Identify the most recent failing wrapper function where available.

Return: Pointer to the requested result, or NULL when unavailable; consult the implementation for ownership.

qscossl_error_set

void qscossl_error_set(qscossl_error_reason reason);

Record a QSCOSSL error code and operation identifier.

Parameters: reason: Value used to configure or select the operation.

Return: No value is returned.

qscossl_error_unsupported

int qscossl_error_unsupported(const char* functionname);

Report a request for an operation outside the implemented subset.

Parameters: functionname: Pointer to caller-supplied data or state; see API ownership conventions.

Return: API-specific status or result; zero commonly indicates failure, except for status/length queries.

19.18 qscossl_extcipher.h (5 declarations)

QSC authenticated RCS and CSX cipher extensions.

QSCOSSL_EXTCIPHER_is_available

int QSCOSSL_EXTCIPHER_is_available(void);

Determine whether QSC RCS/CSX one-shot adapters are compiled.

Return: 1 when QSC extended ciphers are enabled, 0 otherwise; thread-safe.

QSCOSSL_RCS_seal

int QSCOSSL_RCS_seal(qscossl_rcs_profile profile, const uint8_t* key, size_t keylen, const uint8_t* nonce, const uint8_t* info, size_t infolen, const uint8_t* associated, size_t associatedlen, const uint8_t* plaintext, size_t plaintextlen, uint8_t* output, size_t outputcap, size_t* outputlen);

Encrypt and authenticate one RCS record, appending its MAC.

Parameters: profile: RCS-256 or RCS-512 profile.; key: Secret key, keylen bytes (32 or 64 for the selected profile).; keylen: Length of the input secret key.; nonce: Exactly 32 nonce bytes; remains caller-owned and is not modified.; info: Optional QSC key-schedule customization, infolen bytes.; infolen: Customization length; 0 permits NULL info.; associated: Optional authenticated associated data, associatedlen bytes.; associatedlen: Associated-data length, at most UINT32_MAX.; plaintext: Source input, plaintextlen bytes; exact in-place usage is supported.; plaintextlen: Payload size in bytes, including zero.; output: Writable ciphertext and MAC buffer, outputcap bytes.; outputcap: Output buffer capacity, at least plaintextlen plus tag length.; outputlen: Receives ciphertext plus tag size, set to zero on failure.

Return: 1 on successful encryption or 0 on invalid parameters/QSC failure.

Notes: Keys and nonces must be unique per record; do not reuse overlapping counter ranges. Output belongs to caller. Independent calls have no shared state.

QSCOSSL_RCS_open

int QSCOSSL_RCS_open(qscossl_rcs_profile profile, const uint8_t* key, size_t keylen, const uint8_t* nonce, const uint8_t* info, size_t infolen, const uint8_t* associated, size_t associatedlen, const uint8_t* ciphertext, size_t ciphertextlen, uint8_t* output, size_t outputcap, size_t* outputlen);

Authenticate an RCS record before exposing its plaintext.

Parameters: profile: RCS-256 or RCS-512 profile.; key: Secret key, keylen bytes.; keylen: Input key length, 32 or 64 per profile.; nonce: Exactly 32 nonce bytes; remains unchanged.; info: Optional customization, infolen bytes.; infolen: Customization size.; associated: Optional associated data, associatedlen bytes.; associatedlen: Associated-data length.; ciphertext: Ciphertext with trailing MAC, ciphertextlen bytes.; ciphertextlen: Ciphertext length including selected tag length.; output: Caller-owned plaintext buffer, outputcap bytes.; outputcap: Output capacity of at least ciphertextlen minus tag length.; outputlen: Receives verified plaintext size, set zero on failure.

Return: 1 only on verified plaintext, 0 on invalid tag or arguments.

Notes: QSC keeps output unchanged on authentication failure. No unauthenticated data is released.

QSCOSSL_CSX_seal

int QSCOSSL_CSX_seal(const uint8_t* key, size_t keylen, const uint8_t* nonce, const uint8_t* info, size_t infolen, const uint8_t* associated, size_t associatedlen, const uint8_t* plaintext, size_t plaintextlen, uint8_t* output, size_t outputcap, size_t* outputlen);

Encrypt a CSX-512 record and append its 64-byte authentication MAC.

Parameters: key: 64-byte secret key, keylen must equal 64.; keylen: Key length in bytes.; nonce: 16-byte nonce; caller buffer remains unchanged.; info: Optional key-schedule customization, infolen bytes, max 14.; infolen: Customization length (0 permits NULL).; associated: Optional AAD, associatedlen bytes.; associatedlen: Associated-data length.; plaintext: Plaintext input, plaintextlen nonzero bytes.; plaintextlen: Payload length in bytes.; output: Caller-owned ciphertext+MAC output, outputcap bytes.; outputcap: Buffer capacity in bytes.; outputlen: Receives actual ciphertext+MAC size, zero on failure.

Return: 1 on success; 0 on QSC failure or invalid arguments.

Notes: Nonce-key reuse is forbidden. No key or nonce is retained after return.

QSCOSSL_CSX_open

int QSCOSSL_CSX_open(const uint8_t* key, size_t keylen, const uint8_t* nonce, const uint8_t* info, size_t infolen, const uint8_t* associated, size_t associatedlen, const uint8_t* ciphertext, size_t ciphertextlen, uint8_t* output, size_t outputcap, size_t* outputlen);

Verify a CSX-512 tag before releasing its plaintext.

Parameters: key: 64-byte secret key, keylen must equal 64.; keylen: Key length in bytes.; nonce: 16-byte nonce; caller buffer remains unchanged.; info: Optional customization, infolen bytes, max 14.; infolen: Customization length.; associated: Optional AAD, associatedlen bytes.; associatedlen: Associated-data length.; ciphertext: Ciphertext+MAC input, ciphertextlen bytes.; ciphertextlen: Total input length, greater than 64 bytes.; output: Caller-owned plaintext buffer, outputcap bytes.; outputcap: Output capacity in bytes.; outputlen: Receives verified plaintext length, zero on failure.

Return: 1 only on authenticated success, or 0 otherwise.

Notes: QSC leaves plaintext unchanged on authentication failure.

19.19 qscossl_hw.h (4 declarations)

Opaque hardware-backed signing, agreement and attestation.

QSCOSSL_HW_is_available

int QSCOSSL_HW_is_available(void);

Query whether this wrapper was compiled with the QSC hardware facade.

Return: 1 for hardware-facade support or 0 when disabled; thread-safe.

Notes: Does not imply that a provider is configured, a device is present, or FIPS certification.

QSCOSSL_HW_key_sign

qsc_hw_status QSCOSSL_HW_key_sign(qsc_hw_keyref* key, const uint8_t* message, size_t msglen, uint8_t* signature, size_t* signaturelen);

Sign using a caller-owned opaque, hardware-backed QSC key reference.

Parameters: key: Valid QSC hardware key reference; no private bytes are exported.; message: Message buffer, length msglen bytes.; msglen: Number of message bytes.; signature: Caller-owned output buffer, capacity in *signaturelen.; signaturelen: In/out signature buffer capacity and actual output length.

Return: QSC hardware status; caller must inspect errors and may not fall back silently.

Notes: Key lifetime, backend thread safety, and permitted algorithms follow QSC hwprovider contracts.

QSCOSSL_HW_key_agree

qsc_hw_status QSCOSSL_HW_key_agree(qsc_hw_keyref* key, const uint8_t* peer, size_t peerlen, uint8_t* secret, size_t* secretlen);

Perform hardware-backed key agreement without exposing private material.

Parameters: key: Caller-owned QSC hardware key reference.; peer: Public peer input, peerlen bytes.; peerlen: Number of peer public-key bytes.; secret: Caller-owned shared-secret buffer, capacity *secretlen.; secretlen: In/out secret capacity and actual length.

Return: Typed QSC hardware operation status; never silently use a software key.

Notes: Secret output is caller-owned and must be securely erased after use.

QSCOSSL_HW_key_attest

qsc_hw_status QSCOSSL_HW_key_attest(qsc_hw_keyref* key, const uint8_t* challenge, size_t challengelen, uint8_t* evidence, size_t* evidencelen);

Request hardware-key attestation evidence.

Parameters: key: Caller-owned hardware key reference.; challenge: Challenge buffer, challengelen bytes.; challengelen: Challenge length.; evidence: Caller-owned evidence output buffer, capacity *evidencelen.; evidencelen: In/out evidence buffer capacity and actual length.

Return: Typed QSC hardware status; evidence must be validated by the application.

Notes: Key custody and attestation trust depend on the configured backend.

19.20 qscossl_kdf.h (4 declarations)

Explicit HKDF, PBKDF2 and SHAKE interfaces.

QSCOSSL_KDF_hkdf

int QSCOSSL_KDF_hkdf(const EVP_MD* digest, const uint8_t* salt, size_t saltlength, const uint8_t* ikm, size_t ikmlength, const uint8_t* info, size_t infolength, uint8_t* output, size_t outputlength);

Derive an RFC 5869 HKDF output with a selected SHA-2 digest.

Parameters: digest: An EVP_sha256(), EVP_sha384() or EVP_sha512() descriptor.; salt: Optional salt bytes (NULL when saltlength is zero).; saltlength: Salt length in bytes.; ikm: Input key material (NULL only when ikmlength is zero).; ikmlength: Input key material length in bytes.; info: Optional application context (NULL when infolength is zero).; infolength: Context length in bytes.; output: Writable caller-owned output buffer of outputlength bytes.; outputlength: Requested nonzero output length, at most 255 digest blocks.

Return: 1 on success; 0 on error. Output is cleared on failure.

Notes: Thread-safe. Only the output buffer is retained by the caller.

QSCOSSL_KDF_pbkdf2

int QSCOSSL_KDF_pbkdf2(const EVP_MD* digest, const uint8_t* password, size_t passwordlength, const uint8_t* salt, size_t saltlength, uint64_t iterations, uint8_t* output, size_t outputlength);

Derive a PBKDF2 key using QSC PBMAC1, supporting up to 64 bytes.

Parameters: digest: A SHA-256, SHA-384, or SHA-512 EVP descriptor.; password: Password bytes, NULL only when passwordlength is zero.; passwordlength: Password length in bytes.; salt: Salt bytes, NULL only when saltlength is zero.; saltlength: Salt length in bytes.; iterations: Positive iteration count within QSC PBMAC1 limits.; output: Writable, caller-owned output buffer.; outputlength: Requested output length from 1 to QSC_PBMAC1_MAX_KEY_SIZE.

Return: 1 on success; 0 on unsupported lengths or failure.

Notes: Use PKCS5_PBKDF2_HMAC for longer keys. Thread-safe.

QSCOSSL_SHAKE_compute

int QSCOSSL_SHAKE_compute(unsigned int bits, const uint8_t* input, size_t inputlength, uint8_t* output, size_t outputlength);

Compute an extendable-output SHAKE digest using QSC.

Parameters: bits: Security strength, 128 or 256.; input: Data bytes, NULL only when inputlength is zero.; inputlength: Byte length of input.; output: Caller-owned writable buffer.; outputlength: Requested nonzero output bytes.

Return: 1 on success; 0 on invalid input. Thread-safe.

QSCOSSL_cSHAKE_compute

int QSCOSSL_cSHAKE_compute(unsigned int bits, const uint8_t* input, size_t inputlength, const uint8_t* name, size_t namelength, const uint8_t* custom, size_t customlength, uint8_t* output, size_t outputlength);

Compute a customizable cSHAKE digest using QSC.

Parameters: bits: Security strength, 128 or 256.; input: Input bytes, NULL only when inputlength is zero.; inputlength: Input byte count.; name: Optional function-name bytes, NULL if namelength is zero.; namelength: Function-name byte count.; custom: Optional custom string, NULL if customlength is zero.; customlength: Custom-string byte count.; output: Writable caller-owned buffer.; outputlength: Requested nonzero output length.

Return: 1 on success; 0 on invalid parameters. Thread-safe.

19.21 qscossl_mac.h (7 declarations)

KMAC/QMAC/Poly1305 streaming authentication.

QSCOSSL_MAC_CTX_free

void QSCOSSL_MAC_CTX_free(QSCOSSL_MAC_CTX* context);

Frees a QSCOSSL MAC extension context.

Parameters: context: Owned MAC context, freed and erased; NULL may be accepted.

QSCOSSL_MAC_CTX_new

QSCOSSL_MAC_CTX* QSCOSSL_MAC_CTX_new(qscossl_algorithm_id algorithm);

Allocates a QSCOSSL MAC extension context for a QSC-backed MAC algorithm.

Parameters: algorithm: Supported KMAC, QMAC, or Poly1305 selector.

Return: A MAC extension context, or NULL on failure.

QSCOSSL_MAC_CTX_reset

int QSCOSSL_MAC_CTX_reset(QSCOSSL_MAC_CTX* context);

Resets a QSCOSSL MAC extension context while preserving its algorithm.

Parameters: context: Existing MAC context to reset.

Return: 1 on success; 0 on failure.

QSCOSSL_MAC_final

int QSCOSSL_MAC_final(QSCOSSL_MAC_CTX* context, uint8_t* output, size_t* outputlength);

Finalizes a QSCOSSL MAC extension context.

Parameters: context: Initialized MAC context.; output: Writable MAC tag buffer, at least the configured tag length.; outputlength: In/out tag buffer capacity and resulting size in bytes.

Return: 1 on success; 0 on failure.

QSCOSSL_MAC_init

int QSCOSSL_MAC_init(QSCOSSL_MAC_CTX* context, const uint8_t* key, size_t keylength, const uint8_t* custom, size_t customlength, const uint8_t* nonce, size_t noncelength, size_t outputlength);

Initializes a QSCOSSL MAC extension context.

Parameters: context: MAC context allocated for the selected algorithm.; key: Secret MAC key of keylength bytes; copied and erased during cleanup.; keylength: Key size in bytes, validated against algorithm constraints.; custom: Optional MAC customization data; length supplied in customlength.; customlength: Customization size in bytes; zero permits NULL.; nonce: QMAC nonce, or NULL for MAC algorithms without a nonce.; noncelength: Nonce length in bytes.; outputlength: Requested MAC tag size in bytes.

Return: 1 on success; 0 on failure.

QSCOSSL_MAC_size

size_t QSCOSSL_MAC_size(const QSCOSSL_MAC_CTX* context);

Returns the output size of a QSCOSSL MAC extension context.

Parameters: context: MAC context whose configured output size is queried.

Return: The MAC output size in bytes, or zero on failure.

QSCOSSL_MAC_update

int QSCOSSL_MAC_update(QSCOSSL_MAC_CTX* context, const uint8_t* input, size_t inputlength);

Updates a QSCOSSL MAC extension context with message input.

Parameters: context: Initialized MAC context.; input: Message bytes to absorb; may be NULL only when inputlength is zero.

Return: 1 on success; 0 on failure.

19.22 qscossl_mlkem.h (5 declarations)

Runtime-selected ML-KEM encapsulation.

QSCOSSL_MLKEM_is_available

int QSCOSSL_MLKEM_is_available(void);

Report whether independently selectable ML-KEM profiles were linked.

Return: 1 when built with all independent QSC Kyber profile implementations, otherwise 0.

Notes: This is not a cryptographic conformance or FIPS-approval claim. Thread-safe.

QSCOSSL_MLKEM_get_sizes

int QSCOSSL_MLKEM_get_sizes(qscossl_mlkem_profile profile, size_t* publickeylen, size_t* privatekeylen, size_t* ciphertextlen, size_t* secretlen);

Retrieve exact key, ciphertext, and secret sizes for a selected profile.

Parameters: profile: Selected parameter set, including explicitly experimental 1280.; publickeylen: Output for public key size, in bytes.; privatekeylen: Output for private key size, in bytes.; ciphertextlen: Output for encapsulated ciphertext size, in bytes.; secretlen: Output for shared-secret size, in bytes.

Return: 1 when recognized; 0 on an unsupported profile or invalid pointers.

Notes: All outputs are caller-owned; no allocation or shared state.

QSCOSSL_MLKEM_keygen

int QSCOSSL_MLKEM_keygen(qscossl_mlkem_profile profile, uint8_t* publickey, size_t publickeycap, uint8_t* privatekey, size_t privatekeycap);

Generate an ML-KEM key pair using the QSC private RNG provider.

Parameters: profile: Selected parameter set.; publickey: Caller-owned output, at least publickeycap bytes.; publickeycap: Public-key buffer capacity in bytes.; privatekey: Caller-owned secret-key output, at least privatekeycap bytes.; privatekeycap: Secret-key buffer capacity in bytes.

Return: 1 on successful generation; 0 on insufficient capacity or QSC RNG/algorithm failure.

Notes: Erase privatekey securely when no longer needed; no ownership transfer. Thread-safe for disjoint buffers.

QSCOSSL_MLKEM_encapsulate

int QSCOSSL_MLKEM_encapsulate(qscossl_mlkem_profile profile, const uint8_t* publickey, size_t publickeylen, uint8_t* ciphertext, size_t ciphertextcap, uint8_t* secret, size_t secretcap);

Encapsulate a secret to a validated ML-KEM public key.

Parameters: profile: Selected parameter set.; publickey: Public input key, length publickeylen.; publickeylen: Public-key size in bytes; must match profile exactly.; ciphertext: Caller-owned ciphertext output, capacity ciphertextcap.; ciphertextcap: Ciphertext buffer capacity in bytes.; secret: Caller-owned shared-secret output, capacity secretcap.; secretcap: Shared-secret buffer capacity in bytes.

Return: 1 on success; 0 on invalid input, insufficient capacity or QSC failure.

Notes: No inputs and outputs may overlap; generated secret belongs to caller.

QSCOSSL_MLKEM_decapsulate

int QSCOSSL_MLKEM_decapsulate(qscossl_mlkem_profile profile, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* ciphertext, size_t ciphertextlen, uint8_t* secret, size_t secretcap);

Decapsulate a ciphertext using a selected ML-KEM secret key.

Parameters: profile: Selected parameter set.; privatekey: Private input key, length privatekeylen.; privatekeylen: Secret-key size in bytes; must match profile exactly.; ciphertext: Encapsulated input ciphertext, length ciphertextlen.; ciphertextlen: Input ciphertext length; must match profile exactly.; secret: Caller-owned shared-secret output, capacity secretcap.; secretcap: Shared-secret output capacity in bytes.

Return: 1 on valid key and successful decapsulation; 0 on invalid input/key.

Notes: Invalid ciphertext uses QSC's implicit-rejection behavior and may still return 1. Never use this return code as a ciphertext-authenticity indicator. Thread-safe for disjoint buffers.

19.23 qscossl_ocsp.h (1 declarations)

Authenticated offline OCSP response checks.

QSCOSSL_OCSP_check_der

qscossl_ocsp_result QSCOSSL_OCSP_check_der(const uint8_t* response_der, size_t response_len, const uint8_t* issuer_der, size_t issuer_len, const uint8_t* subject_der, size_t subject_len, const char* utcgeneralizedtime, size_t timelen);

Validate a locally supplied, signed OCSP response against the certificate and issuer.

Parameters: response_der: Caller-owned OCSP response DER, up to 8192 bytes, never retained.; response_len: Nonzero number of response DER bytes.; issuer_der: Independently authenticated issuing certificate DER, up to 8192 bytes.; issuer_len: Nonzero issuer certificate DER byte count.; subject_der: Certificate DER whose serial number is checked, up to 8192 bytes.; subject_len: Nonzero subject certificate DER byte count.; utcgeneralizedtime: Exactly 15 ASCII bytes in YYYYMMDDHHMMSSZ format.; timelen: Exactly 15 bytes, excluding an optional terminator.

Return: Good, revoked or unknown only after OCSP responder identity, signature, certificate identifier and update-time verification succeeds; otherwise error.

Notes: The issuer MUST be validated independently by the caller. An authenticated UNKNOWN result is not approval. No external network I/O is performed; the signed response is copied into QSC's transp...

19.24 qscossl_pkcs12.h (6 declarations)

Authenticated PKCS#12/PFX import.

QSCOSSL_PKCS12_import_der

QSCOSSL_PKCS12* QSCOSSL_PKCS12_import_der(const uint8_t* der, size_t derlength, const char* password);

Parse and authenticate a DER-encoded PKCS #12 version 3 bundle.

Parameters: der: Caller-owned PFX DER bytes, not retained.; derlength: Nonzero DER byte count (maximum QSC_X509_PKCS12_DER_MAX).; password: Caller-owned NUL-terminated password (not retained).

Return: Owned parsed bundle or NULL for invalid MAC, malformed DER, or unsupported profile.

Notes: The QSC parser restricts PBES2 encryption and enforces certificate/key matching. Thread-safe for separate inputs; an individual returned handle is not synchronized.

QSCOSSL_PKCS12_free

void QSCOSSL_PKCS12_free(QSCOSSL_PKCS12* bundle);

Erase secrets and free a parsed bundle. \param bundle Owned handle or NULL, without concurrent use.

Parameters: bundle: Owned handle or NULL, without concurrent use.

QSCOSSL_PKCS12_certificate_count

size_t QSCOSSL_PKCS12_certificate_count(const QSCOSSL_PKCS12* bundle);

Read count of certificates in an authenticated bundle.

Parameters: bundle: Imported bundle, caller-owned.

Return: Number of parsed certificates; zero for invalid handles.

QSCOSSL_PKCS12_get_certificate

X509* QSCOSSL_PKCS12_get_certificate(const QSCOSSL_PKCS12* bundle, size_t index);

Extract a deep-copy certificate at the given index.

Parameters: bundle: Imported bundle, caller-owned.; index: Zero-based certificate index less than certificate_count.

Return: Newly owned X509 certificate or NULL; caller frees via X509_free.

Notes: Certificates remain usable after the bundle is freed.

QSCOSSL_PKCS12_has_private_key

int QSCOSSL_PKCS12_has_private_key(const QSCOSSL_PKCS12* bundle);

Determine whether the bundle contains a validated private key.

Parameters: bundle: Imported bundle, caller-owned.

Return: 1 for an available decoded private key, 0 otherwise.

QSCOSSL_PKCS12_get_private_key

EVP_PKEY* QSCOSSL_PKCS12_get_private_key(const QSCOSSL_PKCS12* bundle);

Import a supported key into a QSCOSSL EVP_PKEY from an authenticated bundle.

Parameters: bundle: Imported bundle, caller-owned.

Return: Newly owned EVP_PKEY or NULL for unsupported curves/key types or absent key.

Notes: Only the active QSC ECDSA curve and supported EdDSA types are mapped. This function never exports the private key to the caller as raw bytes.

19.25 qscossl_pqc.h (3 declarations)

Post-quantum key encapsulation extensions.

QSCOSSL_KEM_decapsulate

int QSCOSSL_KEM_decapsulate(EVP_PKEY* key, uint8_t* secret, size_t* secretlen, const uint8_t* ciphertext, size_t ciphertextlen);

Decapsulate a supplied ciphertext using the selected private KEM key.

Parameters: key: Key material or key handle.; secret: Pointer to caller-supplied data or state; see API ownership conventions.; secretlen: Size of secret in bytes.; ciphertext: Pointer to caller-supplied data or state; see API ownership conventions.; ciphertextlen: Ciphertext size in bytes.

QSCOSSL_KEM_encapsulate

int QSCOSSL_KEM_encapsulate(EVP_PKEY* key, uint8_t* ciphertext, size_t* ciphertextlen, uint8_t* secret, size_t* secretlen);

Encapsulate a new secret to the selected KEM public key.

Parameters: key: Key material or key handle.; ciphertext: Pointer to caller-supplied data or state; see API ownership conventions.; ciphertextlen: Ciphertext size in bytes.; secret: Pointer to caller-supplied data or state; see API ownership conventions.; secretlen: Size of secret in bytes.

QSCOSSL_KEM_keygen

int QSCOSSL_KEM_keygen(EVP_PKEY* key);

Generate a QSC-backed public/private KEM key pair.

Parameters: key: Key material or key handle.

20: Implementation and Release Checklist

The following checklist constitutes the minimum documented handoff for an application consuming the QSCOSSL wrapper. It is not a certification statement.

- The application's OpenSSL call inventory is compared against the actual QSCOSSL public header/implementation surface.
- All binary and source dependencies are pinned, and ABI-incompatible OpenSSL objects are not linked into the wrapper target.
- Selected QSC build profile, AES-NI/AVX targeting, compiler switches and runtime are recorded and consistent.
- Every cryptographic key size, nonce, tag length, buffer capacity, status and error path has a documented test.
- Decryption with authenticated output uses QSCOSSL_EVP_DecryptFinal_ex2, with negative tests for short buffers and bad tags.
- All certificates are trust-validated before use, and the application explicitly handles revocation UNKNOWN/ERROR.
- Native blocking TCP ownership and TLS shutdown have been tested on the target operating system.
- Unimplemented OpenSSL APIs fail safely and are blocked by a clear support policy.
- The appropriate standard or provider mode has a separate, independently reproducible test matrix.
- Release evidence includes passing native builds and tests, sanitizer reports where supported and protected packaging.

Annex A: Data Types and Constants

A.1 Opaque compatibility type families

The qscossl_types.h header forward-declares BIO, EVP_CIPHER, EVP_CIPHER_CTX, EVP_MD, EVP_MD_CTX, EVP_PKEY, EVP_PKEY_CTX, HMAC_CTX, QSCOSSL_MAC_CTX, SSL_CTX, SSL_METHOD, SSL, X509, X509_STORE_CTX and X509_STORE as compatibility handle types. Construct and release them through matching APIs; their sizes and binary layouts are not a public contract.

A.2 Selected enums and constants

Family	Named examples	Engineering note
Random provider	qscossl_random_provider_csp=1; _acp=2	The default production recommendation is CSP.
AEAD controls	EVP_CTRL_GCM_SET_IVLEN, GET_TAG, SET_TAG	Call in the correct order around update and finalization.
Digest limits	EVP_MAX_MD_SIZE = 64	Actual digest output is queried or defined by the algorithm.
Cipher buffer limits	EVP_MAX_KEY_LENGTH = 64, EVP_MAX_IV_LENGTH = 32	These storage upper limits do not imply every size is valid for every cipher.
ML-KEM profiles	512, 768, 1024, experimental 1280	1280 is not a standardized ML-KEM parameter set.
RCS profiles	qscossl_rcs_256 / qscossl_rcs_512	Keys and tags differ by selected profile.
CRL result	ERROR=-1, NOT_REVOKED=0, REVOKED=1	ERROR is not a good/clear response.
OCSP result	ERROR=-1, GOOD=0, REVOKED=1, UNKNOWN=2	UNKNOWN is not authorization.
Binary compatibility	QSCOSSL_BINARY_COMPATIBILITY = 0	Source-compatible subset, not an OpenSSL ABI replacement.

Table A.1 Selected public metadata

Annex B: Repository and Test Index

B.1 Public header map

Header	API count	Role
openssl/bio.h	4	Public compatibility declarations / selectors
openssl/crypto.h	15	Public compatibility declarations / selectors
openssl/err.h	6	Public compatibility declarations / selectors
openssl/evp.h	68	Public compatibility declarations / selectors
openssl/hmac.h	7	Public compatibility declarations / selectors
openssl/kdf.h	0	Public compatibility declarations / selectors
openssl/pem.h	4	Public compatibility declarations / selectors
openssl/rand.h	6	Public compatibility declarations / selectors
openssl/ssl.h	41	Public compatibility declarations / selectors
openssl/x509.h	18	Public compatibility declarations / selectors
qscossl.h	4	QSC extension or configuration types
qscossl_aead.h	1	QSC extension or configuration types
qscossl_cert_lifecycle.h	4	QSC extension or configuration types
qscossl_config.h	0	QSC extension or configuration types
qscossl_crl.h	1	QSC extension or configuration types
qscossl_csr.h	2	QSC extension or configuration types
qscossl_errors.h	7	QSC extension or configuration types
qscossl_extcipher.h	5	QSC extension or configuration types
qscossl_hw.h	4	QSC extension or configuration types
qscossl_kdf.h	4	QSC extension or configuration types
qscossl_mac.h	7	QSC extension or configuration types
qscossl_mlkem.h	5	QSC extension or configuration types
qscossl_ocsp.h	1	QSC extension or configuration types
qscossl_pkcs12.h	6	QSC extension or configuration types
qscossl_pqc.h	3	QSC extension or configuration types
qscossl_types.h	0	QSC extension or configuration types

Table B.1 Public source headers in this snapshot

B.2 Test and documentation location

The supplied QSCOSSLTest/ tree contains stage-based tests and optional loopback suites; the genuine OpenSSL provider has separate test targets. Review the CMake configurations and dated qualification records for actual platform coverage. Historical stage results do not establish universal cross-platform success.